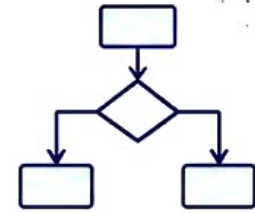
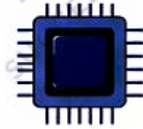


1010101010
0101010101
1010101010
0101010101



Infographics



Visual Learning

Better understanding through visual content.



Simplified Concepts

Complex topics explained in simple and visual form.



Quick Revision

Easy to revise important points and diagrams.

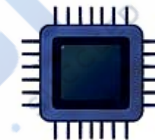
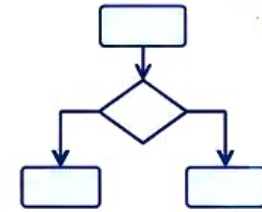


Exam Ready

Helps in faster revision and better exam preparation.



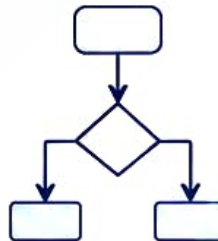
1010101010
0101010101
1010101010
0101010101



Unit-I



10110
01001
10101





A string is a sequence of characters enclosed in **single quotes** `' '` or **double quotes** `" "`

BASIC STRING OPERATIONS IN PYTHON

Example

```
s = "Hello, Python!"
```

Index : 0 1 2 3 4 5 6 7 8 9 10 11 12 13

Char : H e l l o , P y t h o n !

1. INDEXING

Accessing characters using **index** (starting from 0).

```
s = "Hello, Python!"
```

Index : 0 1 2 3 4 5 6 7 8 9 10 11 12 13
 ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
 -14 -13 -12 -11 -10 -9 -8 -7 -5 -4 -3 -2 -1

```
s[0]    # 'H' (first character)
s[7]    # 'P'
s[-1]   # '!' (last character)
s[-6]   # 'y'
```

2. SLICING

Extract a part of the string using start:end:step

Syntax: `s[start:end:step]`

- **start** → starting index (inclusive) [default = 0]
- **end** → ending index (exclusive) [default = len(s)]
- **step** → step size [default = 1]

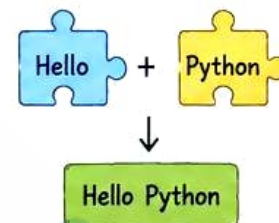
```
s = "Hello, Python!"
s[0:5]    # 'Hello'
s[7:13]   # 'Python'
s[:5]     # 'Hello' (start to index 4)
s[7:]     # 'Python!' (from index 7 to end)
s[-6:]    # 'Python!' (last 6 characters)
s[::2]    # 'Hloyhn' (every 2nd character)
s[::-1]   # '!nohtyP,olleH' (reverse string)
```

3. STRING CONCATENATION

Joining two or more strings using **+** operator.

```
a = "Hello"
b = "Python"
c = a + " " + b
```

```
print(c) # Hello Python
```



4. STRING REPETITION

Repeating a string using ***** operator.

```
s = "Hi!"
print(s * 3)    # Hi!Hi!Hi!
print("-" * 10) # -----
```

Repeat
n times

5. MEMBERSHIP OPERATOR

Check if a substring exists in a string.

```
s = "Hello, Python!"
```

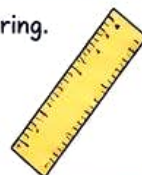
```
"Hello" in s    # True
"Python" in s   # True
"Java" in s     # False
```



6. LENGTH

Find number of characters in a string.

```
s = "Hello, Python!"
print(len(s)) # 14
```



7. ESCAPE SEQUENCES

Used to represent special characters in a string.

Sequence	Meaning	Sequence	Meaning
<code>\n</code>	New Line	<code>\'</code>	Single Quote (')
<code>\t</code>	Tab	<code>\"</code>	Double Quote (")
<code>\\</code>	Backslash (\)	<code>\r</code>	Carriage Return

Example :

```
print("Hello\nPython")
```

Output:
Hello
Python

QUICK SUMMARY



- ✓ Indexing → Access single character
- ✓ Slicing → Extract a part of string
- ✓ Concatenation → Join strings

- ✓ Repetition → Repeat strings
- ✓ Membership → Check presence of substring
- ✓ Length → Find total characters

- ✓ Escape Sequences → Represent special characters

TIP

Strings in Python are **IMMUTABLE**, which means once created, they cannot be changed.



A string is a sequence of characters. Each character in a string has a unique position called an **index**.

STRING INDEXING & SLICING IN PYTHON

```
s = "Hello, Python!"
```

Index : 0 1 2 3 4 5 6 7 8 9 10 11 12 13
Char : H e l l o , P y t h o n !

1. STRING INDEXING

Accessing a single character from a string using its index.

- Positive index starts from 0 (left to right)
- Negative index starts from -1 (right to left)

```
s = "Hello, Python!"
```

Positive Index : 0 1 2 3 4 5 7 8 9 10 11 12 13
↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
Characters : H e l l o , y t h o n !
Negative Index : -14 -13 -12 -11 -10 -9 -8 -7 -6 -5 -4 -3 -2 -1

Examples

```
s = "Hello, Python!"
s[0]    # 'H' (first character)
s[7]    # 'P'
s[-1]   # '!' (last character)
s[-6]   # 'P'
s[13]   # '!' (same as s[-1])
s[-14]  # 'H' (same as s[0])
```

Note:

If you try to access an index outside the range, Python will raise an **IndexError**.

```
print(s[100])
# IndexError:
# string index out of range
```

2. STRING SLICING

Extracting a part (substring) from a string using **[start : end : step]**.

- start → starting index (inclusive) [default = 0]
- end → ending index (exclusive) [default = len(s)]
- step → step size / jump [default = 1]

Examples

Expression	Visual (index)	Result	Explanation
s[0:5]	0 1 2 3 4 5 6 7 8 9 10 11 12 13 ↓ ↓ ↓ ↓ ↓ H e l l o , P y t h o n !	'Hello'	From index 0 to 4
s[7:13]	0 1 2 3 4 5 6 7 8 9 10 11 12 13 ↓ ↓ ↓ ↓ ↓ P y t h o n	'Python'	From index 7 to 12
s[:5]	0 1 2 3 4 5 6 7 8 9 10 11 12 13 ↓ ↓ ↓ ↓ ↓ H e l l o , P y t h o n !	'Hello'	Start to index 4
s[7:]	0 1 2 3 4 5 6 7 8 9 10 11 12 13 ↓ ↓ ↓ ↓ ↓ P y t h o n !	'Python!'	From index 7 to end
s[:]	0 1 2 3 4 5 6 7 8 9 10 11 12 13 ↓ ↓ ↓ ↓ ↓ H e l l o , P y t h o n !	'Hello, Python!'	Whole string
s[::2]	0 1 2 3 4 5 6 7 8 9 10 11 12 13 ↓ ↓ ↓ ↓ ↓ H l o P t o !	'HloPto!'	Every 2nd character
s[::-1]	13 12 11 10 9 8 7 6 5 4 3 2 1 0 ↓ ↓ ↓ ↓ ↓ ! n o h t y P , o o l l e H	'!nohtyP ,olleH'	Reverse string

3. SLICING WITH NEGATIVE INDEX

Expression	Result	Explanation
s[-6:-1]	'Python'	From index -6 to -2
s[-7:]	'Python!'	From index -7 to end
s[:-7]	'Hello, '	Start to index -8
s[-6:-1:2]	'Pto'	From -6 to -2, step 2

QUICK SUMMARY

- ★ Indexing → Access single character: `s[i]`
- ★ Slicing → Access substring: `s[start:end:step]`
- ★ Start is inclusive, End is exclusive.
- ★ Default start = 0, Default end = len(s), Default step = 1
- ★ Step can be negative (useful for reversing).

TIPS

- Use len(s) to get the length of the string.
- Use negative index to count from the end.
- Slicing never raises **IndexError**; it returns an empty string if out of range.





What is String Formatting?

String formatting is the process of embedding variables or values inside a string.

STRING FORMATTING IN PYTHON

★ String formatting lets us insert variables or values inside a string in a readable way. ★

Example

```
name = "Saquib"
age = 20

msg = "My name is Saquib and my age is 20."
print(msg)
```

METHODS OF STRING FORMATTING

1. Old Style Formatting (%)

- Uses % operator.
- Placeholders start with % followed by a format character.

Format Characters

Specifier	Meaning
%s	String
%d	Integer
%f	Float
%c	Character

Example

```
name = "Saquib"
age = 20
height = 5.8
msg = "Name: %s, Age: %d, Height: %.1f" %
      (name, age, height)
print(msg)
```

Output

```
Name: Saquib, Age: 20, Height: 5.8
```

Note:

The number of placeholders must match the number of values.

2. str.format() Method

- Uses the str.format() function.
- Uses curly braces {} as placeholders.

Example

```
name = "Saquib"
age = 20
height = 5.8
msg = "Name: {}, Age: {}, Height: {:.1f}".format(
      name, age, height)
print(msg)
```

Output

```
Name: Saquib, Age: 20, Height: 5.8
```

Using Index Numbers

You can refer to values by their position.

```
msg = "{2} is {0} years old and {1} tall.".format(
      age, height, name)
print(msg)
```

Output

```
Saquib is 20 years old and 5.8 tall.
```

3. f-Strings (Formatted String Literals)

- Introduced in Python 3.6+.
- Uses f or F prefix before the string.
- Place variables directly inside {}.

Example

```
name = "Saquib"
age = 20
height = 5.8
msg = f"Name: {name}, Age: {age}, Height: {height:.1f}"
print(msg)
```

Output

```
Name: Saquib, Age: 20, Height: 5.8
```

Advantages

- ✓ Most readable and concise.
- ✓ Fast and recommended way.
- ✓ Supports expressions also.

Example with expression

```
price = 499
tax = 0.18
total = f"Total price: ₹{price + price*tax:.2f}"
print(total)
```

Output

```
Total price: ₹588.82
```

4. Template Strings (string.Template)

- Uses \$ sign followed by variable name.
- Requires the template module.

Example

```
from string import Template
t = Template("Name: $name, Age: $age")
msg = t.substitute(name="Saquib", age=20)
print(msg)
```

Output

```
Name: Saquib, Age: 20
```

Features Comparison

Method	Syntax Style	Readable	Performance
% Formatting	% operator	Medium	Slow
str.format()	{ } and .format()	Good	Medium
f-string	f"...{var}..."	Excellent	Fast
Template	\$variable	Good	Slow

FORMAT SPECIFIERS (for % and str.format())

Specifier	Meaning	Example	Output
%s / {}	String	"%s" % "Hello"	Hello
%d / {}	Integer	"%d" % 25	25
%f / {}	Float (default 6 decimal places)	"%f" % 3.14159	3.141590
%.2f / {:.2f}	Float with 2 decimal places	"%.2f" % 3.14159	3.14

Specifier	Meaning	Example	Output
%c / {}	Character	"%c" % 'A'	A
%x / {:x}	Hexadecimal	"%x" % 255	ff
%o / {:o}	Octal	"%o" % 8	10
%%	Literal %	"100%% sure"	100% sure

TIPS

- 💡 Use f-strings for modern Python (3.6+).
- 💡 Use .format() when you need more control.
- 💡 Use % for older code or very simple formatting.
- 💡 Always match placeholders with correct values.





In Python, strings are **immutable**, which means string methods do not change the original string. They return a new string.

COMMON STRING METHODS IN PYTHON

String methods are functions that perform operations on strings. They are called using dot (.) operator.

EXAMPLE

```
s = "Hello, Python!"
print(s.upper()) # HELLO, PYTHON!
print(s)         # Hello, Python! (unchanged)
```

1. CASE CONVERSION METHODS

Method	Description	Example	Output
<code>lower()</code>	Converts all characters to lowercase.	"HeLLo".lower()	'hello'
<code>upper()</code>	Converts all characters to uppercase.	"HeLLo".upper()	'HELLO'
<code>capitalize()</code>	Converts first character to uppercase, rest lowercase.	"hello world".capitalize()	'Hello world'
<code>title()</code>	Converts first character of each word to uppercase.	"hello world".title()	'Hello World'
<code>swapcase()</code>	Swaps lowercase to uppercase and vice versa.	"HeLLo".swapcase()	'hELLo'
<code>casefold()</code>	Converts string to lower case (suitable for comparisons).	"Straße".casefold()	'strasse'

2. SEARCH & FIND METHODS

Method	Description	Example	Output
<code>find(sub)</code>	Returns lowest index of substring, -1 if not found.	"Hello".find('e')	1
<code>rfind(sub)</code>	Returns highest index of substring, -1 if not found.	"banana".rfind('a')	5
<code>index(sub)</code>	Same as find(), but raises ValueError if not found.	"Hello".index('e')	1
<code>rindex(sub)</code>	Same as rfind(), but raises ValueError if not found.	"banana".rindex('a')	5
<code>count(sub)</code>	Returns count of occurrences of substring.	"banana".count('a')	3

3. CHECKING METHODS

Method	Description	Example	Output
<code>startswith(prefix)</code>	Returns True if string starts with prefix.	"Hello".startswith('He')	True
<code>endswith(suffix)</code>	Returns True if string ends with suffix.	"Hello".endswith('lo')	True
<code>isalpha()</code>	Returns True if all characters are alphabets.	"Python".isalpha()	True
<code>isdigit()</code>	Returns True if all characters are digits.	"12345".isdigit()	True
<code>isalnum()</code>	Returns True if all characters are alphanumeric (letters + digits).	"Py123".isalnum()	True
<code>isspace()</code>	Returns True if all characters are whitespace.	"\t\n".isspace()	True

4. MODIFICATION METHODS

Method	Description	Example	Output
<code>replace(old, new[, count])</code>	Replaces old substring with new substring.	"a-b-c".replace('-', '+')	'a+b+c'
<code>strip([chars])</code>	Removes leading and trailing whitespace (or specified chars).	" hello ".strip()	'hello'
<code>lstrip([chars])</code>	Removes leading whitespace (or chars).	" hello".lstrip()	'hello'
<code>rstrip([chars])</code>	Removes trailing whitespace (or chars).	"hello ".rstrip()	'hello'
<code>removeprefix(prefix)</code>	Removes prefix if present (Python 3.9+).	"HelloWorld".removeprefix('Hello')	'World'
<code>removesuffix(suffix)</code>	Removes suffix if present (Python 3.9+).	"filename.txt".removesuffix('.txt')	'filename'
<code>center(width[, fillchar])</code>	Centers string in given width, padded with fillchar.	"hi".center(6, '*')	'**hi**'
<code>ljust(width[, fillchar])</code>	Left justifies string in given width.	"hi".ljust(6, '*')	'hi*****'
<code>rjust(width[, fillchar])</code>	Right justifies string in given width.	"hi".rjust(6, '*')	'*****hi'
<code>zfill(width)</code>	Pads string on the left with zeros to make it of width.	"42".zfill(5)	'00042'

5. SPLIT & JOIN METHODS

Method	Description	Example	Output
<code>split(sep)</code>	Splits string by separator sep (default is any whitespace).	"a,b,c".split(',')	['a', 'b', 'c']
<code>rsplit(sep)</code>	Splits from right by separator.	"a,b,c,d".rsplit(',', 1)	['a,b,c', 'd']
<code>splitlines()</code>	Splits string at line breaks.	"a\b\nc".splitlines()	['a', 'b', 'c']
<code>join(iterable)</code>	Joins elements of iterable with the string as separator.	"-".join(['a', 'b', 'c'])	'a-b-c'

6. OTHER USEFUL METHODS

Method	Description	Example	Output
<code>len(s)</code>	Returns length of string.	<code>len("Hello")</code>	5
<code>min(s)</code>	Returns smallest character (lexicographically).	<code>min("Python")</code>	'P'
<code>max(s)</code>	Returns largest character (lexicographically).	<code>max("Python")</code>	'y'
<code>ord(c)</code>	Returns Unicode value of character.	<code>ord('A')</code>	65
<code>chr(n)</code>	Returns character for Unicode value n.	<code>chr(65)</code>	'A'

IMPORTANT NOTES

- Strings are **immutable** – methods return new strings.
- Methods do not change the original string.
- Most methods are case-sensitive.
- Some methods (find, index) are very useful for searching.
- Use `help(str)` in Python shell to see all string methods.



EXTRA EXAMPLES

```
s = " Hello, Python World! "
print(s.lower()) # " hello, python world! "
print(s.strip().title()) # "Hello, Python World!"
print(s.replace('World', 'Students')) # " Hello, Python Students! "
print(s.split(',')) # [" Hello", " Python World! "]
print('-'.join(['Python', 'is', 'fun'])) # 'Python-is-fun'
```



A list in Python is an **ordered**, **mutable** collection of items. Lists can store items of different data types.

CREATING LISTS IN PYTHON



There are multiple ways to create lists in Python.



Example

```
fruits = ['apple', 'banana', 'cherry']
numbers = [1, 2, 3, 4, 5]
mixed = [1, 'hello', 3.14, True]
print(fruits)           # ['apple', 'banana', 'cherry']
```

1. DIFFERENT WAYS TO CREATE LISTS

Method	Syntax	Example	Output
1. Using square brackets [] (Most common way)	<code>list_name = [item1, item2, item3, ...]</code>	<code>colors = ['red', 'green', 'blue']</code>	<code>['red', 'green', 'blue']</code>
2. Using list() constructor	<code>list_name = list(iterable)</code>	<code>nums = list((1, 2, 3, 4))</code>	<code>[1, 2, 3, 4]</code>
3. Using range() with list()	<code>list_name = list(range(start, stop, step))</code>	<code>even_nums = list(range(2, 11, 2))</code>	<code>[2, 4, 6, 8, 10]</code>
4. From a string (each character as item)	<code>list_name = list('string')</code>	<code>chars = list('Python')</code>	<code>['P', 'y', 't', 'h', 'o', 'n']</code>
5. List comprehension	<code>list_name = [expression for item in iterable]</code>	<code>squares = [x*x for x in range(1, 6)]</code>	<code>[1, 4, 9, 16, 25]</code>
6. Using * operator (repetition)	<code>list_name = [item] * n</code>	<code>zeros = [0] * 5</code>	<code>[0, 0, 0, 0, 0]</code>

2. CREATING AN EMPTY LIST

An empty list has no items.

- Method 1 : Using empty square brackets

```
empty_list1 = []           # Output: []
```

- Method 2 : Using list() constructor

```
empty_list2 = list()       # Output: []
```



NOTE

- Lists can hold items of different data types.
- Lists are ordered (index starts from 0).
- Lists are mutable (their items can be changed).

3. EXAMPLES OF LISTS WITH DIFFERENT DATA TYPES

List	Code	Output	Description
List of Integers	<code>a = [1, 2, 3, 4, 5]</code>	<code>[1, 2, 3, 4, 5]</code>	All items are integers
List of Strings	<code>b = ['cat', 'dog', 'bird']</code>	<code>['cat', 'dog', 'bird']</code>	All items are strings
List of Floats	<code>c = [1.1, 2.2, 3.3]</code>	<code>[1.1, 2.2, 3.3]</code>	All items are floats
Mixed List	<code>d = [1, 'hello', 3.14, True]</code>	<code>[1, 'hello', 3.14, True]</code>	Different data types
Nested List	<code>e = [[1, 2], [3, 4], [5, 6]]</code>	<code>[[1, 2], [3, 4], [5, 6]]</code>	List containing lists

4. MINI PROGRAM EXAMPLES

```
# Create a list of first 5 natural numbers
nums = list(range(1, 6))
print(nums)
# Output: [1, 2, 3, 4, 5]
```

```
# Create a list of vowels from a string
word = 'education'
vowels = [ch for ch in word if ch in 'aeiou']
print(vowels)
# Output: ['e', 'u', 'a', 'i', 'o']
```

QUICK SUMMARY



- Use [] to create lists.
- Use list() to convert other iterables (tuple, string, range) to list.
- Use list(range()) to generate number sequences.

- Use list comprehension for concise list creation.
- Use * operator to repeat items in a list.
- Empty list can be created using [] or list().

Remember!

Lists are one of the most powerful and commonly used data structures in Python.





A list is an **ordered, mutable** collection of items.

List operations allow us to **access, modify, add, remove** or organize list elements.

LIST OPERATIONS IN PYTHON



Lists in Python are mutable, which means we can change, add or remove items.



EXAMPLE LIST

Let's consider a list

`L = [10, 20, 30, 40, 50]`

No.	Operation	Description	Syntax	Example	Output
1.	Accessing Elements	Access elements using index (index starts from 0)	<code>L[index]</code>	<code>L[2]</code>	30
2.	Negative Indexing	Access elements from the end (-1 refers to last element)	<code>L[-1]</code>	<code>L[-1]</code>	50
3.	Slicing	Extract a part (sub-list) from the list	<code>L[start : end : step]</code>	<code>L[1:4]</code>	[20, 30, 40]
4.	Concatenation	Join two or more lists	<code>L1 + L2</code>	<code>[1,2] + [3,4]</code>	[1, 2, 3, 4]
5.	Repetition	Repeat the list <i>n</i> number of times	<code>L * n</code>	<code>L * 2</code>	[10, 20, 30, 40, 50, 10, 20, 30, 40, 50]
6.	Length	Find number of elements in the list	<code>len(L)</code>	<code>len(L)</code>	5
7.	Membership	Check if an element exists in the list	<code>x in L / x not in L</code>	<code>30 in L</code>	True
8.	Minimum / Maximum	Find smallest / largest element	<code>min(L) / max(L)</code>	<code>min(L)</code> <code>max(L)</code>	10 50
9.	Sum	Find sum of all numeric elements	<code>sum(L)</code>	<code>sum(L)</code>	150
10.	Sorted List	Return a sorted copy of list	<code>sorted(L)</code>	<code>sorted(L, reverse=True)</code>	[50, 40, 30, 20, 10]
11.	Reversed List	Return reversed copy of list	<code>L[::-1]</code> or <code>list(reversed(L))</code>	<code>L[::-1]</code>	[50, 40, 30, 20, 10]
12.	Copy List	Create a copy of list	<code>L.copy()</code> or <code>L[:]</code>	<code>L.copy()</code>	[10, 20, 30, 40, 50]
13.	Clear List	Remove all items from the list	<code>L.clear()</code>	<code>L.clear()</code>	[]
14.	Delete List	Delete the entire list	<code>del L</code>	<code>del L</code>	L no longer exists

COMMON ADD / REMOVE OPERATIONS

Method	Description	Example	Output
<code>append(x)</code>	Add element <i>x</i> at the end	<code>L.append(60)</code>	[10, 20, 30, 40, 50, 60]
<code>insert(i, x)</code>	Insert <i>x</i> at index <i>i</i>	<code>L.insert(2, 25)</code>	[10, 20, 25, 30, 40, 50]
<code>extend(iterable)</code>	Add all elements from iterable	<code>L.extend([60,70])</code>	[10, 20, 30, 40, 50, 60, 70]
<code>remove(x)</code>	Remove first occurrence of <i>x</i>	<code>L.remove(30)</code>	[10, 20, 40, 50]
<code>pop([i])</code>	Remove & return item at index <i>i</i> (default last item)	<code>L.pop()</code> <code>L.pop(1)</code>	50 [10, 30, 40, 50] (returned 20)
<code>del L[i]</code>	Delete item at index <i>i</i>	<code>del L[1]</code>	[10, 30, 40, 50]
<code>clear()</code>	Remove all items	<code>L.clear()</code>	[]



KEY POINTS

Lists are ordered and mutable (changeable).

- Indexing starts from 0.
- Negative indexing starts from -1 (last element).
- Slicing does not include the end index.
- Many operations return a new list and do not change the original list.

QUICK SUMMARY



- ✓ Access elements → `L[i]`
- ✓ Extract part → `L[start:end:step]`
- ✓ Add elements → `append()`, `insert()`, `extend()`
- ✓ Remove elements → `remove()`, `pop()`, `del`
- ✓ Other useful operations → `len()`, `min()`, `max()`, `sum()`, `sorted()`, `reversed()`, `copy()`, `clear()`

IMPORTANT NOTES



- Use `append()` to add at the end (fast operation).
- Use `insert()` when you need to add at a specific position.
- Use `pop()` carefully; it removes and returns the element.
- Use `clear()` to empty the list without deleting the list itself.



A list is an **ordered**, **mutable** collection of items.

Indexing helps to access a single element.

Slicing helps to access a sub-list (part of a list).

INDEXING & SLICING IN LISTS

Lists are ordered collections. We can access elements using index and extract parts using slicing.

EXAMPLE LIST

L = ['A', 'B', 'C', 'D', 'E', 'F', 'G']

Positive Index : 0 1 2 3 4 5 6

Negative Index : -7 -6 -5 -4 -3 -2 -1

1. INDEXING IN LISTS

Accessing a single element using its index.

- Index starts from 0 (left to right).
- Negative index starts from -1 (right to left).

L = ['A', 'B', 'C', 'D', 'E', 'F', 'G']

A	B	C	D	E	F	G
↓	↓	↓	↓	↓	↓	↓
0	1	2	3	4	5	6
-7	-6	-5	-4	-3	-2	-1

Examples

Expression	Explanation	Result
L[0]	First element	'A'
L[3]	Element at index 3	'D'
L[6]	Last element	'G'
L[-1]	Last element	'G'
L[-3]	Third last element	'E'
L[2]	Element at index 2	'C'



If index is out of range, Python raises **IndexError**.

Example : L[10] # IndexError

2. SLICING IN LISTS

Extracting a part (sub-list) from a list.

Syntax : L[start : end : step]

- **start** → starting index (inclusive) [default = 0]
- **end** → ending index (exclusive) [default = len(L)]
- **step** → step size / jump [default = 1]

SLICING EXAMPLES

Expression	Visual (index)	Result	Explanation
L[1:4]	0 1 2 3 4 5 6 A B C D E F G	['B', 'C', 'D']	From index 1 to 3 (4-1)
L[:4]	0 1 2 3 4 5 6 A B C D E F G	['A', 'B', 'C', 'D']	Start to index 3 (default start = 0)
L[3:]	0 1 2 3 4 5 6 A B C D E F G	['D', 'E', 'F', 'G']	From index 3 to end (default end)
L[:]	0 1 2 3 4 5 6 A B C D E F G	['A', 'B', 'C', 'D', 'E', 'F', 'G']	Whole list (default start, end, step)
L[::2]	0 1 2 3 4 5 6 A B C D E F G	['A', 'C', 'E', 'G']	Step of 2
L[1::2]	0 1 2 3 4 5 6 A B C D E F G	['B', 'D', 'F']	Start 1, step 2
L[::-1]	6 5 4 3 2 1 0 G F E D C B A	['G', 'F', 'E', 'D', 'C', 'B', 'A']	Reverse the list
L[-4:-1]	-7 -6 -5 -4 -3 -2 -1 D E F	['D', 'E', 'F']	From -4 to -2 (-1 is excluded)

3. SLICING WITH STEP (ADVANCED)

Step can be positive or negative.

Expression	Result	Explanation
L[::3]	['A', 'D', 'G']	Every 3rd element starting from 0
L[1:6:2]	['B', 'D', 'F']	From 1 to 5, step 2
L[5:1:-1]	['F', 'E', 'D', 'C']	From 5 down to 2, step -1
L[::-2]	['G', 'E', 'C', 'A']	Reverse with step 2

4. IMPORTANT POINTS

- Start index is inclusive, End index is exclusive.
- Default start = 0, Default end = len(L), Default step = 1.
- Negative index is used to count from the end.
- Step can be negative (reverse slicing).

If we try to access an invalid range → empty list is returned.

Example : L[10:20] → []

QUICK SUMMARY



- ✓ Use L[index] to access a single element.
- ✓ Use L[start:end] to extract a part of the list.
- ✓ Start is included, End is excluded.
- ✓ Use step to jump through the list.

HANDY EXAMPLES

L = [10, 20, 30, 40, 50, 60, 70]

L[0] → 10
L[-1] → 70
L[2:5] → [30, 40, 50]
L[:3] → [10, 20, 30]

L[4:] → [50, 60, 70]
L[::2] → [10, 30, 50, 70]
L[1:2] → [20, 40, 60]
L[::-1] → [70, 60, 50, 40, 30, 20, 10]

TIP

Use indexing for single items.
Use slicing for sub-lists.
Practice combinations to become faster!





A list is an **ordered**, **mutable** collection of items.

BUILT-IN FUNCTIONS ON LISTS AND LIST METHODS IN PYTHON

Lists in Python have a set of built-in functions and methods that make it easy to work with and manipulate data.

Example List

```
L = [10, 20, 30, 40, 50]
Index : 0   1   2   3   4
        -5  -4  -3  -2  -1
```

1. BUILT-IN FUNCTIONS ON LISTS

These functions can be used with lists (and other iterables).

No.	Function	Description	Syntax	Example	Output
1.	<code>len()</code>	Returns number of elements in the list	<code>len(L)</code>	<code>len([10, 20, 30])</code>	3
2.	<code>min()</code>	Returns smallest element	<code>min(L)</code>	<code>min([10, 20, 30])</code>	10
3.	<code>max()</code>	Returns largest element	<code>max(L)</code>	<code>max([10, 20, 30])</code>	30
4.	<code>sum()</code>	Returns sum of all numeric elements	<code>sum(L)</code>	<code>sum([10, 20, 30])</code>	60
5.	<code>sorted()</code>	Returns a new sorted list (ascending)	<code>sorted(L)</code>	<code>sorted([30, 10, 20])</code>	[10, 20, 30]
6.	<code>reversed()</code>	Returns a reverse iterator of the list	<code>reversed(L)</code> Or for x in <code>reversed([10,20,30])</code> :	<code>list(reversed([10, 20, 30]))</code>	[30, 20, 10]
7.	<code>sum()</code> with start	Returns sum of list elements + start	<code>sum(L, start)</code>	<code>sum([10, 20, 30], 5)</code>	65
8.	<code>any()</code>	Returns True if any element is True (non-zero for numbers)	<code>any(L)</code>	<code>any([0, 0, 5, 0])</code>	True
9.	<code>all()</code>	Returns True if all elements are True (non-zero for numbers)	<code>all(L)</code>	<code>all([1, 2, 3])</code>	True
10.	<code>enumerate()</code>	Returns index and element as pairs	<code>enumerate(L)</code>	<code>list(enumerate(['a','b','c']))</code>	[(0, 'a'), [1, 'b'), [2, 'c')]
11.	<code>zip()</code>	Combines elements from multiple lists (pair-wise)	<code>zip(L1,L2,...)</code>	<code>list(zip([1,2,3],['a','b','c']))</code>	[(1, 'a'), [2, 'b'), [3, 'c')]
12.	<code>map()</code>	Applies a function to all elements of list/iterable	<code>map(func, L)</code>	<code>list(map(lambda x: x*2, [1,2,3]))</code>	[2, 4, 6]

2. LIST METHODS

These methods are called on the list object. They modify the list (most of them).

No.	Method	Description	Syntax	Example (L = [10, 20, 30])	Result (L)
1.	<code>append(x)</code>	Add element x at the end	<code>L.append(40)</code>		[10, 20, 30, 40]
2.	<code>extend(iterable)</code>	Add all elements from iterable at the end	<code>L.extend([40, 50])</code>		[10, 20, 30, 40, 50]
3.	<code>insert(i, x)</code>	Insert element x at index i	<code>L.insert(1, 15)</code>		[10, 15, 20, 30]
4.	<code>remove(x)</code>	Remove first occurrence of element x	<code>L.remove(20)</code>		[10, 30]
5.	<code>pop([i])</code>	Remove and return element at index i (default last)	<code>L.pop()</code> <code>L.pop(0)</code>	# last # at index 0	30 10 [20]
6.	<code>clear()</code>	Remove all items from the list	<code>L.clear()</code>		[]
7.	<code>index(x[, start[, end]])</code>	Return first index of x (start & end optional)	<code>L.index(30)</code>		2
8.	<code>count(x)</code>	Return number of times x appears in list	<code>L.count(10)</code>		1
9.	<code>sort(*, key=None, reverse=False)</code>	Sort the list in ascending order	<code>L.sort()</code> <code>L.sort(reverse=True)</code>	# Ascending	[10, 20, 30] [30, 20, 10]
10.	<code>reverse()</code>	Reverse the list in place	<code>L.reverse()</code>		[30, 20, 10]
11.	<code>copy()</code>	Return a shallow copy of the list	<code>L.copy()</code>		[10, 20, 30]
12.	<code>del L[i]</code> or <code>del L[i:j]</code>	Delete element(s) at index i or slice	<code>del L[1]</code> <code>del L[0:2]</code>	# delete 20 # delete 10,20	[10, 30] [30]
13.	<code>in / not in</code>	Check membership in the list	<code>10 in L</code> → True <code>50 not in L</code> → True		True True
14.	<code>* operator</code>	Repeat the list n number of times	<code>L * 2</code>		[10,20,30, 10,20,30]



KEY POINTS

- Built-in functions do not change the original list.
- Most list methods modify the list in place.
- list is mutable (changeable).
- Use functions for calculations / info.
- Use methods to modify the list.

EXAMPLES (BUILT-IN FUNCTIONS)

```
L = [3, 1, 4, 1, 5]
len(L)           # 5
min(L), max(L)   # (1, 5)
sorted(L)        # [1, 1, 3, 4, 5]
reversed(L)      # reverse iterator
sum(L)           # 14
any(L), all(L)   # (True, True)
enumerate(L)     # (0,3), (1,1), (2,4) ...
zip(L, ['a','b','c','d','e'])
# [(3, 'a'), (1, 'b'), (4, 'c'), ...]
```

EXAMPLES (LIST METHODS)

```
L = [1, 2, 3]
L.append(4)      # [1, 2, 3, 4]
L.extend([5, 6]) # [1, 2, 3, 4, 5, 6]
L.insert(2, 99)  # [1, 2, 99, 3, 4, 5, 6]
L.remove(99)     # [1, 2, 3, 4, 5, 6]
L.pop()          # returns 6 → [1, 2, 3, 4, 5]
L.sort()         # [1, 2, 3, 4, 5]
L.reverse()      # [5, 4, 3, 2, 1]
L.clear()        # []
```



TIP

Use `help(list)` in Python shell for all built-in functions and list methods.



PYTHON DICTIONARIES: CREATING DICTIONARIES

A **Dictionary** in Python is an unordered collection of data values, used to store data values in **key : value** pairs.



KEY POINTS

- Keys must be unique and immutable (string, number, tuple).
- Values can be of any data type.
- Dictionaries are mutable (changeable).



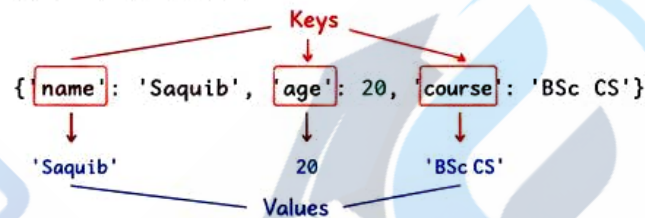
1. CREATING DICTIONARIES

We can create dictionaries in several ways.

a) Using curly braces {} (most common)

```
student = {'name': 'Saqib', 'age': 20, 'course': 'BSc CS'}
```

Output / Representation



b) Using dict() constructor

```
person = dict(name='Ali', age=21, city='Akola')
```

c) From a list of tuples

```
marks = dict([('Math', 95), ('Physics', 90), ('Chemistry', 88)])
```

d) Using dictionary comprehension

```
squares = {x: x*x for x in range(1, 6)}  
# Output: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

2. EXAMPLES

Example 1: Using curly braces

```
student = {'name': 'Saqib', 'age': 20, 'course': 'BSc CS'}  
print(student)  
# Output: {'name': 'Saqib', 'age': 20, 'course': 'BSc CS'}
```

Example 2: Using dict() constructor

```
person = dict(name='Ali', age=21, city='Akola')  
print(person)  
# Output: {'name': 'Ali', 'age': 21, 'city': 'Akola'}
```

Example 3: From list of tuples

```
marks = dict([('Math', 95), ('Physics', 90), ('Chemistry', 88)])  
print(marks)  
# Output: {'Math': 95, 'Physics': 90, 'Chemistry': 88}
```

Example 4: Using dictionary comprehension

```
squares = {x: x*x for x in range(1, 6)}  
print(squares)  
# Output: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

3. EMPTY DICTIONARIES

a) Using empty braces

```
empty_dict = {}  
print(empty_dict) # Output: {}
```

b) Using dict() constructor

```
empty_dict = dict()  
print(empty_dict) # Output: {}
```

c) For specific data type (optional)

```
from collections import defaultdict  
count = defaultdict(int)  
print(count) # Output: defaultdict(<class 'int'>, {})
```

4. VALID AND INVALID EXAMPLES

Valid Dictionary

```
{'id': 1, 'name': 'Aman'}  
  
{1: 'one', 2: 'two'}  
  
{(1,2): 'tuple key', 3: [1,2,3]}
```

Invalid Dictionary (Reason)

```
{'id': 1, 'id': 2}  
(Duplicate keys not allowed)  
  
[{'list': 10}]  
(List is mutable, not allowed as key)  
  
[{'dict': 1}: 10]  
(Dictionary is mutable, not allowed as key)
```

5. NESTED DICTIONARY (EXAMPLE)

```
student_info = {  
    'student1': {'name': 'Saqib', 'age': 20},  
    'student2': {'name': 'Ali', 'age': 21}  
}  
print(student_info)  
# Output:  
# {'student1': {'name': 'Saqib', 'age': 20},  
#  'student2': {'name': 'Ali', 'age': 21}}
```

6. SUMMARY

- ✓ Dictionaries store data in **key : value** pairs.
- ✓ Keys must be unique and immutable.
- ✓ Values can be of any data type.
- ✓ Dictionaries are unordered and mutable.
- ✓ We can create dictionaries using {}, dict(), list of tuples, or dictionary comprehension.



7. QUICK REFERENCE

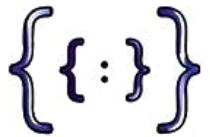
Method	Usage	Example	Output
Create with {}	{key: value, ...}	{ 'a': 1, 'b': 2 }	{ 'a': 1, 'b': 2 }
Create with dict()	dict(key=value, ...)	dict(x=1, y=2)	{ 'x': 1, 'y': 2 }
From list of tuples	dict([(k1,v1), (k2,v2)])	dict([('a',1), ('b',2)])	{ 'a': 1, 'b': 2 }
Dict Comprehension	{k:v for ... in ...}	{x:x*x for x in range(3)}	{0: 0, 1: 1, 2: 4}



Remember: Use dictionaries when you want to store related data with meaningful keys for easy access and readability.



Accessing & Modifying Key:Value Pairs in Dictionaries



Dictionaries store data in **key:value** pairs. Keys are unique, values can be any data type.



ACCESSING KEY:VALUE PAIRS

1 Access value by key

Use the key inside square brackets []

```
d = {'name': 'Aman', 'age': 20, 'city': 'Akola'}
value = d['name'] # 'Aman'
```

Output
'Aman'

2 Using get() method

Returns the value for the key. Returns None (or default) if key is not present.

```
d = {'name': 'Aman', 'age': 20}
d.get('age') # 20
d.get('city') # None
d.get('city', 'Not Found') # 'Not Found'
```

Output
20
None
'Not Found'

3 Access all keys, values or items

Use keys(), values(), items()

```
d = {'name': 'Aman', 'age': 20}
d.keys() # dict_keys(['name', 'age'])
d.values() # dict_values([20])
d.items() # dict_items([('name', 'Aman'), ('age', 20)])
```

Returns
view
objects

4 Check if a key exists

Use in operator

```
d = {'name': 'Aman', 'age': 20}
'name' in d # True
'city' in d # False
```

Output
True
False

QUICK EXAMPLE

```
# Create dictionary
d = {'name': 'Aman', 'age': 20}
# Access
d['name'] # 'Aman'
d.get('age') # 20
# Modify
d['age'] = 21
d['city'] = 'Akola'
# Delete
del d['name']
# Final dictionary
d # {'age': 21, 'city': 'Akola'}
```

★ KEY POINTS

- Keys must be unique and immutable (str, int, tuple, etc.)
- Values can be any data type and can be duplicated.
- Access by key using [] or get().
- Modify by assignment, update() or del/pop().
- Be careful while deleting—key must exist (use pop() with default to avoid errors).



MODIFYING KEY:VALUE PAIRS

1 Add a new key:value pair

Assign a value to a new key.

```
d = {'name': 'Aman', 'age': 20}
d['city'] = 'Akola'
# {'name': 'Aman', 'age': 20, 'city': 'Akola'}
```

New pair
is added ✓

2 Modify an existing value

Assign a new value to an existing key.

```
d = {'name': 'Aman', 'age': 20}
d['age'] = 21
# {'name': 'Aman', 'age': 21}
```

Value
updated ✓

3 Update multiple key:value pairs

Use update() method or | operator (Python 3.9+)

```
d = {'name': 'Aman'}
d.update({'age': 20, 'city': 'Akola'})
# {'name': 'Aman', 'age': 20, 'city': 'Akola'}
d |= {'course': 'BCA'}
# {'name': 'Aman', 'age': 20, 'city': 'Akola', 'course': 'BCA'}
```

Dictionary
merged /
updated ✓

4 Delete a key:value pair

Use del or pop()

```
d = {'name': 'Aman', 'age': 20, 'city': 'Akola'}
del d['city'] # removes the key 'city'
age = d.pop('age') # removes and returns value
# d -> {'name': 'Aman'}, age -> 20
```

Pair
removed ✓

⚙️ COMMON METHODS

- d.keys() → all keys
- d.values() → all values
- d.items() → all key:value pairs
- d.get(key[, default]) → get value safely
- d.update(other_dict) → add/modify pairs
- d.pop(key[, default]) → remove & return value
- d.popitem() → remove last inserted pair
- d.clear() → remove all pairs



💡 TIPS

- Prefer get() when the key might not exist.
- Use in to check key existence before access.
- update() is useful to merge dictionaries.
- Dictionaries are unordered (in versions <3.7). From Python 3.7+, insertion order is preserved.





Built-in Functions on DICTIONARIES



★ Python provides several built-in functions that work with dictionaries. ★ ★ ★

Dictionary =
collection of
key:value pairs



FUNCTION	DESCRIPTION	SYNTAX	EXAMPLE	OUTPUT
 len(d)	Returns the number of key:value pairs in the dictionary.	<code>len(dictionary)</code>	<pre># Example d = {'a': 10, 'b': 20, 'c': 30} len(d)</pre>	3
 str(d)	Returns a string representation of the dictionary.	<code>str(dictionary)</code>	<pre>d = {'a': 10, 'b': 20} str(d)</pre>	<pre>{'a': 10, 'b': 20}</pre>
 dict()	Creates a dictionary from an iterable of key:value pairs.	<code>dict(iterable)</code> or <code>dict(**kwargs)</code>	<pre># From list of tuples dict([('a', 1), ('b', 2)]) # From keyword arguments dict(a=1, b=2)</pre>	<pre>{'a': 1, 'b': 2} {'a': 1, 'b': 2}</pre>
 list(d)	Returns a list of dictionary keys.	<code>list(dictionary)</code>	<pre>d = {'a': 10, 'b': 20, 'c': 30} list(d)</pre>	<pre>['a', 'b', 'c']</pre>
 tuple(d)	Returns a tuple of dictionary keys.	<code>tuple(dictionary)</code>	<pre>d = {'a': 10, 'b': 20, 'c': 30} tuple(d)</pre>	<pre>('a', 'b', 'c')</pre>
 set(d)	Returns a set of dictionary keys.	<code>set(dictionary)</code>	<pre>d = {'a': 10, 'b': 20, 'c': 30} set(d)</pre>	<pre>{'a', 'b', 'c'}</pre>
 min(d) max(d)	Returns the minimum or maximum key in the dictionary.	<code>min(dictionary)</code> <code>max(dictionary)</code>	<pre>d = {'a': 10, 'b': 20, 'c': 30} min(d) max(d)</pre>	<pre>'a' 'c'</pre>
 sorted(d)	Returns a sorted list of dictionary keys.	<code>sorted(dictionary)</code>	<pre>d = {'b': 20, 'a': 10, 'c': 30} sorted(d)</pre>	<pre>['a', 'b', 'c']</pre>
 any(d)  all(d)	Returns True if any or all keys are true (non-empty).	<code>any(dictionary)</code> <code>all(dictionary)</code>	<pre>d1 = {'a': 0, 'b': ''} any(d1) all(d1)</pre>	<pre>False False</pre>

QUICK EXAMPLE

```
d = {'name': 'Aman', 'age': 20, 'city': 'Akola'}

len(d)           # 3
list(d)           # ['name', 'age', 'city']
tuple(d)          # ('name', 'age', 'city')
set(d)            # {'name', 'age', 'city'}
str(d)            # {'name': 'Aman', 'age': 20, 'city': 'Akola'}
min(d)            # 'age'
max(d)            # 'name'
sorted(d)         # ['age', 'city', 'name']
```

KEY NOTES

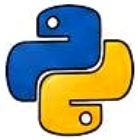
- Most functions operate on dictionary KEYS.
- Values are not considered by functions like `min()`, `max()`, `sorted()`.
- Dictionaries are unordered (Python 3.7+ preserves insertion order, but it's not guaranteed by all operations).
- Use `list(d.values())` to get all values.
- Use `list(d.items())` to get key:value pairs.

RELATED METHODS (Not Functions)

- `d.keys()` → all keys
- `d.values()` → all values
- `d.items()` → all key:value pairs
- `d.get(key, default)` → get value
- `d.update(other_dict)` → merge/update



Tip: Functions are useful for retrieving information. Methods are useful for performing actions on the dictionary.



Dictionary in Python
stores data in
key : value pairs.

PYTHON DICTIONARIES

Dictionary Methods & del Statement



Dictionaries are
mutable, unordered
collections of
key : value pairs.



DICTIONARY METHODS

Dictionary methods are built-in functions used to perform different operations on dictionaries.

METHOD	DESCRIPTION	SYNTAX	EXAMPLE	OUTPUT / RESULT
keys()	Returns a view object containing all the keys.	d.keys()	d = {'a': 1, 'b': 2} d.keys()	dict_keys(['a', 'b'])
values()	Returns a view object containing all the values.	d.values()	d = {'a': 1, 'b': 2} d.values()	dict_values([1, 2])
items()	Returns a view object containing key:value pairs as tuples.	d.items()	d = {'a': 1, 'b': 2} d.items()	dict_items([('a', 1), ('b', 2)])
get(key, default=None)	Returns the value for the specified key. Returns default if key not found.	d.get(key, default)	d = {'a': 1} d.get('a') d.get('x', 'Not Found')	1 Not Found
update(other_dict)	Adds new key:value pairs or updates existing ones.	d.update(other_dict)	d = {'a': 1} d.update({'b': 2})	{'a': 1, 'b': 2}
pop(key, default=None)	Removes the key and returns its value. If key not found, returns default (or error).	d.pop(key, default)	d = {'a': 1, 'b': 2} d.pop('a')	1 # d becomes {'b': 2}
popitem()	Removes and returns the last inserted key:value pair as a tuple.	d.popitem()	d = {'a': 1, 'b': 2} d.popitem()	('b', 2) # d becomes {'a': 1}
clear()	Removes all key:value pairs from the dictionary.	d.clear()	d = {'a': 1, 'b': 2} d.clear()	{} # d becomes empty
copy()	Returns a shallow copy of the dictionary.	d.copy()	d1 = {'a': 1} d2 = d1.copy()	d2 = {'a': 1}
setdefault(key, default)	Returns the value if key exists. If not, inserts the key with default value and returns it.	d.setdefault(key, default)	d = {'a': 1} d.setdefault('b', 10)	10 # d becomes {'a': 1, 'b': 10}

EXAMPLE: USING SOME METHODS

```
student = {'name': 'Aman', 'age': 20}
print(student.keys())    # all keys
print(student.values())  # all values
print(student.get('name')) # get value
student.update({'age': 21}) # update existing
print(student)           # updated dict
```

OUTPUT

```
dict_keys(['name', 'age'])
dict_values(['Aman', 20])
Aman
{'name': 'Aman', 'age': 21}
```

NOTE

- ★ keys(), values(), items() return view objects, not lists. Convert using list() if needed.
- ★ Dictionaries are mutable (changeable).



DEL STATEMENT

The **del** statement is used to delete a key:value pair, a complete dictionary, or other objects.

1 Delete a key:value pair

```
student = {'name': 'Aman', 'age': 20, 'city': 'Akola'}
del student['age']    # remove key 'age'
print(student)
```

OUTPUT

```
{'name': 'Aman',
'city': 'Akola'}
```



Removes the specified key and its value from the dictionary.

2 Delete the complete dictionary

```
student = {'name': 'Aman', 'age': 20}
del student
# Now 'student' no longer exists.
# print(student) -> NameError: name 'student' is not defined
```

The entire
dictionary object
is deleted from
memory.

3 Delete using del with variables and items

```
x = 10
del x    # deletes variable x

lst = [1, 2, 3]
del lst[1]    # deletes item at index 1
print(lst)
```

OUTPUT

```
[1, 3]
```

IMPORTANT DIFFERENCES

OPERATION	WHAT IT DOES	EXAMPLE	RESULT
del d['key']	Deletes a key:value pair	del d['age']	Pair removed
d.pop('key')	Deletes the pair & returns its value	d.pop('age')	Returns value of 'age' and removes the pair
d.clear()	Removes all pairs from the dictionary	d.clear()	Dictionary becomes empty {} (still exists)
del d	Deletes the entire dictionary	del d	Dictionary no longer exists



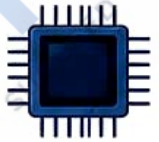
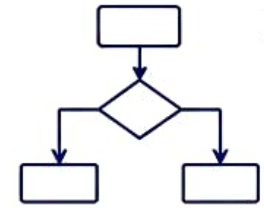
QUICK TIPS

- ✓ Use del d['key'] to remove only specific pair.
- ✓ Use d.pop('key') when you also need the value.
- ✓ Use d.clear() to empty the dictionary.
- ✓ Use del d to remove the dictionary completely.



Master these methods and del statement to handle dictionaries efficiently in Python!

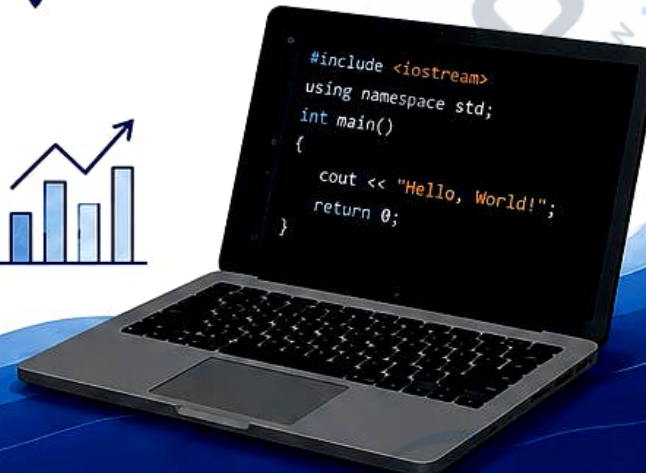
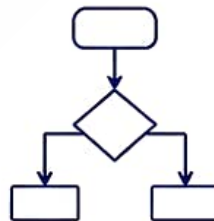
1010101010
0101010101
1010101010
0101010101

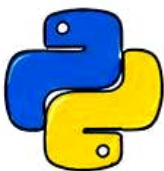


Unit - II



10110
01001
10101





Tuples in Python : Creating Tuples & Basic Tuple Operations

A tuple is an **ordered, immutable** collection of items. It can store elements of different data types.

(1, 2, 3)
Tuple

1. Creating Tuples

Tuples are created by placing comma-separated values inside parentheses ().

1.1 Using literals

```
t1 = (1, 2, 3, 4)
print(t1)
```

Output:
(1, 2, 3, 4)

1.2 Empty tuple

```
t2 = ()
print(t2)
```

Output:
()

1.3 Tuple with one element

```
t3 = (5,)
print(type(t3))
```

Output:
<class 'tuple'>

Note:
Use a comma for single element tuple. (5) is not a tuple, it's just an integer.

1.4 Tuple with different data types

```
t4 = (10, "Hello", 3.5, True)
print(t4)
```

Output:
(10, 'Hello', 3.5, True)

1.5 Using the tuple() constructor

```
t5 = tuple([1, 2, 3, 4])
print(t5)
```

Output:
(1, 2, 3, 4)



KEY POINTS:

- Tuples are written with ().
- Elements are separated by commas.
- They are immutable (cannot be changed after creation).
- They can store different data types.
- Use tuple() to create a tuple from an iterable (like list, string, etc.).

2. Basic Tuple Operations

Tuples support many useful operations. Here are the most common ones:

2.1 Accessing Elements (Indexing)

Use index to get an element. Indexing starts from 0.

```
t = (10, 20, 30, 40, 50)
print(t[0]) # first element
print(t[2]) # third element
```

Output:
10
30

2.3 Concatenation

Join two tuples using +.

```
t1 = (1, 2, 3)
t2 = (4, 5, 6)
t3 = t1 + t2
print(t3)
```

Output:
(1, 2, 3, 4, 5, 6)

2.6 Length

Find the number of elements using len().

```
t = (10, 20, 30, 40)
print(len(t))
```

Output:
4



Remember:

- Tuples are immutable, so you cannot change, add or remove elements.
- But you can create new tuples using operations like +, *, slicing, etc.
- Tuple operations are faster than lists and are useful for fixed data (like coordinates, dates, etc.).

2.2 Slicing

Get a range of elements using slicing.

```
t = (10, 20, 30, 40, 50)
print(t[1:4]) # elements at index 1,2,3
print(t[:3]) # first 3 elements
print(t[2:]) # from index 2 to end
```

Output:
(20, 30, 40)
(10, 20, 30)
(30, 40, 50)

2.4 Repetition

Repeat a tuple using *.

```
t = (1, 2, 3)
t2 = t * 3
print(t2)
```

Output:
(1, 2, 3, 1, 2, 3, 1, 2, 3)

2.7 Min / Max

Get the smallest or largest element.

```
t = (5, 2, 9, 1, 7)
print(min(t))
print(max(t))
```

Output:
1 9

2.5 Membership Test

Check if an element exists using in / not in.

```
t = (10, 20, 30, 40)
print(20 in t)
print(100 in t)
```

Output:
True
False

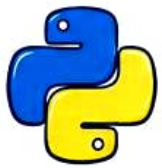
2.8 Tuple Unpacking

Assign elements to variables.

```
t = (10, 20, 30)
a, b, c = t
print(a, b, c)
```

Output:
10 20 30

Tuples are simple,
fast and secure!



Indexing & Slicing in Tuples

Tuples are ordered, immutable collections. We can access elements using indexing and extract a part of the tuple using slicing (same as lists).



Remember:

- Indexing & slicing work in tuples just like in lists.
- Tuples are immutable, so we cannot change elements.

1. Tuple and Indexing

Each element in a tuple has an index (starting from 0).

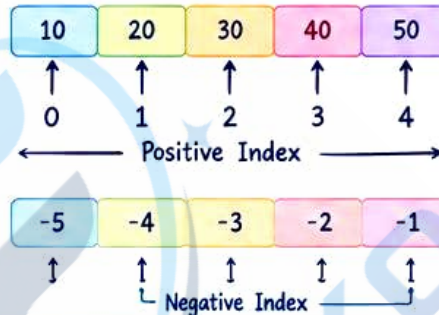
Example:

```
t = (10, 20, 30, 40, 50)
print(t[0]) # first element
print(t[2]) # third element
print(t[-1]) # last element
```

Output:

```
10
30
50
```

Tuple t = (10, 20, 30, 40, 50)



3. Slicing with Step

We can use a step value to skip elements while slicing.

```
t = (10, 20, 30, 40, 50, 60)
print(t[::2]) # every 2nd element
print(t[1::2]) # start from index 1, step 2
print(t[::-1]) # reverse the tuple
```

Output:

```
(10, 30, 50)
(20, 40, 60)
(60, 50, 40, 30, 20, 10)
```

Visual Representation

t[::2] (every 2nd element)
(10, 20, 30, 40, 50, 60)
10 30 50

t[1::2] (start 1, step 2)
(10, 20, 30, 40, 50, 60)
20 40 60

t[::-1] (reverse)
(60, 50, 40, 30, 20, 10)

2. Slicing in Tuples

Slicing returns a new tuple containing a part of the original tuple.

2.1 Basic Slicing

```
t = (10, 20, 30, 40, 50)
print(t[1:4]) # elements at index 1,2,3
print(t[:3]) # first 3 elements
print(t[2:]) # from index 2 to end
```

Output:

```
(20, 30, 40)
(10, 20, 30)
(30, 40, 50)
```

2.2 Using Negative Indices

```
t = (10, 20, 30, 40, 50)
print(t[-3:]) # Last 3 elements
print(t[:-2]) # from start to 2nd last
print(t[-4:-1]) # 4th last to 2nd last
```

Output:

```
(30, 40, 50)
(10, 20, 30)
(20, 30, 40)
```

4. Important Points

- Indexing returns a single element (not a tuple).
- Slicing returns a new tuple (even if it has one element).
- Tuples are immutable, so we cannot assign a new value to an index or slice.
- Slicing works with positive and negative indices.
- Step value can be used to skip elements.
- If the slice range is out of bounds, it does not raise an error (it just gives available elements).

5. More Examples

1. Single element (negative index)

```
t = (5, 15, 25, 35, 45)
print(t[-2])
```

Output:

```
35
```

2. Slice with negative step

```
t = (1, 2, 3, 4, 5)
print(t[::-2])
```

Output:

```
(5, 3, 1)
```

3. Slice till the end

```
t = (100, 200, 300, 400)
print(t[2:])
```

Output:

```
(300, 400)
```

4. Slice from start to index 3

```
t = (11, 22, 33, 44, 55)
print(t[:3])
```

Output:

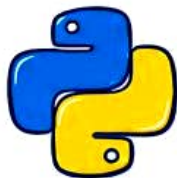
```
(11, 22, 33)
```



Quick Recap

- Indexing: t[index] → single element
- Slicing: t[start:stop:step] → new tuple
- Negative indices: count from the end
- Step: skip elements
- Immutable: cannot modify elements

Same syntax as lists,
just with tuples!



Built-in Functions on Tuples

Tuples are immutable, but we can use many built-in functions to work with them. These functions help us perform useful operations like finding length, counting, sorting, etc.



Remember:

- Tuples are immutable
- Most list functions (like append, remove) don't work on tuples
- But many built-in functions work perfectly with tuples!

1. len()

Returns the number of elements in a tuple.

```
t = (10, 20, 30, 40, 50)
print(len(t))
```

Output:
5

2. type()

Returns the type of the object.

```
t = (1, 2, 3)
print(type(t))
```

Output:
<class 'tuple'>

3. max()

Returns the largest element.

```
t = (10, 25, 7, 40, 3)
print(max(t))
```

Output:
40

4. min()

Returns the smallest element.

```
t = (10, 25, 7, 40, 3)
print(min(t))
```

Output:
3

5. sum()

Returns the sum of numeric elements.

```
t = (10, 20, 30, 40)
print(sum(t))
```

Output:
100

6. sorted()

Returns a new sorted list (tuples are not modified).

```
t = (5, 3, 8, 1)
print(sorted(t))
```

Output:
[1, 3, 5, 8]

7. any()

Returns True if at least one element is True (or non-zero).

```
t = (0, "", 5, False)
print(any(t))
```

Output:
True

8. all()

Returns True if all elements are True (or non-zero).

```
t = (1, 2, 3, 4)
print(all(t))
```

Output:
True

9. enumerate()

Returns an enumerate object (index + value).

```
t = (10, 20, 30)
for i, v in enumerate(t):
    print(i, v)
```

Output:
0 10
1 20
2 30

10. zip()

Combines elements from multiple tuples.

```
t1 = (1, 2, 3)
t2 = ('a', 'b', 'c')
print(list(zip(t1, t2)))
```

Output:
[(1, 'a'), (2, 'b'), (3, 'c')]

11. reversed()

Returns a reverse iterator.

```
t = (10, 20, 30, 40)
print(tuple(reversed(t)))
```

Output:
(40, 30, 20, 10)

12. index()

Returns the index of the first occurrence of a value.

```
t = (10, 20, 30, 20, 10)
print(t.index(20))
```

Output:
1

★ Key Points to Remember

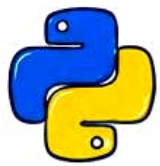
- Tuples are immutable, so functions like append(), remove() or sort() (in-place) are not available.
- Functions like len(), max(), min(), sum(), sorted(), etc. work perfectly with tuples.
- sorted() returns a new list (does not change the original tuple).
- enumerate(), zip(), reversed() are very useful for looping and processing tuples.

Example : Using Multiple Functions

```
t = (4, 1, 7, 2, 9)
print(len(t))           # 5
print(max(t))           # 9
print(min(t))           # 1
print(sum(t))           # 23
print(sorted(t))        # [1, 2, 4, 7, 9]
print(tuple(reversed(t))) # (9, 7, 2, 1, 4)
```

Tuples + Built-in Functions
= Easy Data Handling





Python

Same structure...
Different behavior!



Tuples vs Lists



Both are used to store multiple items, but they are different in a few important ways.
Let's understand the difference with examples.



Remember:

- Lists are mutable (can change).
- Tuples are immutable (cannot change).
- Both can store mixed data types.
- Both support indexing, slicing and many built-in functions.

1. Quick Introduction

List

A list is an ordered collection of items, enclosed in square brackets [].

```
fruits = ["apple", "banana",  
          "mango", 10, 20]
```

Output:

```
['apple', 'banana', 'mango', 10, 20]
```

→ Mutable (can be changed)

Tuple

A tuple is an ordered collection of items, enclosed in round brackets ().

```
fruits = ("apple", "banana",  
          "mango", 10, 20)
```

Output:

```
('apple', 'banana', 'mango', 10, 20)
```

→ Immutable (cannot be changed)

2. Key Differences

Feature	List	Tuple
Syntax	[]	()
Mutability	Mutable	Immutable
Reassignment of elements	Allowed	Not allowed
Methods	Many (append, remove, etc.)	Few (count, index, etc.)
Performance	Slightly slower (more flexible)	Slightly faster (less overhead)
Use case	When you need to change data	When data should remain constant

3. Example Comparison

List Example

```
fruits = ["apple", "banana", "mango"]  
fruits[0] = "grape"
```

Output:

```
['grape', 'banana', 'mango']
```

✓ Works (list is mutable)

List Methods

```
fruits = ["apple", "banana", "mango"]  
fruits.append("orange")  
print(fruits)
```

Output:

```
['apple', 'banana', 'mango', 'orange']
```

Tuple Example

```
fruits = ("apple", "banana", "mango")  
fruits[0] = "grape"
```

TypeError: 'tuple' object does not support item assignment

✗ Error (tuple is immutable)

Tuple Methods

```
fruits = ("apple", "banana", "mango")  
print(fruits.count("mango"))  
print(fruits.index("banana"))
```

Output:

```
1  
1
```

4. More Examples

1. Indexing & Slicing

List

```
nums = [10, 20, 30, 40, 50]  
print(nums[1])      # 20  
print(nums[1:4])    # [20, 30, 40]
```

Output:

```
20  
[20, 30, 40]
```

Tuple

```
nums = (10, 20, 30, 40, 50)  
print(nums[1])      # 20  
print(nums[1:4])    # (20, 30, 40)
```

Output:

```
20  
(20, 30, 40)
```

2. Built-in Functions (both support)

```
nums = [1, 2, 3, 4]  
t = (1, 2, 3, 4)  
print(len(nums))    # 4  
print(len(t))       # 4  
print(sum(nums))    # 10  
print(sum(t))       # 10  
print(max(nums))    # 4  
print(max(t))       # 4  
print(min(nums))    # 1  
print(min(t))       # 1
```



Most built-in functions work on both lists and tuples.

5. List Methods (not for tuples)

```
nums = [1, 2, 3]  
nums.append(4)      # [1, 2, 3, 4]  
nums.remove(2)      # [1, 3, 4]  
nums.insert(1, 10)  # [1, 10, 3, 4]  
nums.pop()          # 4  
nums.sort()         # [1, 3, 10]  
nums.reverse()      # [10, 3, 1]
```



These methods are only available for lists, not tuples.

6. Tuple Methods (very few)

```
t = (1, 2, 3, 2, 4)  
print(t.count(2))   # 2  
print(t.index(3))   # 2
```



Only count() and index() are available for tuples.

7. When to Use What?

Use a List when:

- ✓ You need to add, remove or modify items.
- ✓ You need many built-in methods.
- ✓ You are working with dynamic data.

Use a Tuple when:

- ✓ You want data to remain constant.
- ✓ You need better performance (slightly).
- ✓ You are using it as a key in a dictionary (or set).
- ✓ You want to protect data from changes.



Key Takeaway: Both lists and tuples are useful. The choice depends on whether you need to modify the data or keep it fixed.

Right tool
for the right job!



Same
Language...
Different
Purpose...

Tuples vs Dictionaries

Both are built-in data types in Python, but they are different in structure, usage and behavior.



Remember:

- Tuples = ordered + immutable
- Dictionaries = unordered (in <3.7) / insertion ordered (in 3.7+) + mutable.

Tuples

1. What is a Tuple?

- A tuple is an ordered collection of items.
- It is immutable (cannot be changed).
- Elements are enclosed in round brackets ().

Example:

```
t = (10, 20, 30, "Hello", 5.5)
print(t)
```

Output:

```
(10, 20, 30, 'Hello', 5.5)
```

2. Accessing Elements

- Use index (like a list).

```
t = (10, 20, 30, 40, 50)
print(t[0]) # first element
print(t[-1]) # last element
```

Output:

```
10
50
```

3. Tuple Operations

- Indexing & slicing
- len(), count(), index()
- Concatenation (+) and repetition (*)

```
t1 = (1, 2, 3)
t2 = (4, 5)
print(t1 + t2) # (1, 2, 3, 4, 5)
print(t1 * 2) # (1, 2, 3, 1, 2, 3)
```

Output:

```
(1, 2, 3, 4, 5)
(1, 2, 3, 1, 2, 3)
```



Key Differences (In Short)

- Tuple = ordered + immutable + uses () + index access.
- Dictionary = key-value pairs + mutable + uses { } + key access.

Tuple = Position

Dictionary = Meaning

Quick Comparison

Feature	Tuples	Dictionaries
Structure	Ordered collection of items	Key : value pairs
Syntax	()	{ }
Mutability	Immutable (cannot change)	Mutable (can change)
Ordering	Maintains order (of elements)	Unordered (<3.7) / Insertion order (3.7+)
Access	By index (0, 1, 2, ...)	By key ("name", "age", ...)
Duplicates	Allowed	Keys must be unique (values can repeat)
Use case	Fixed data, read-only data, returning multiple values	Storing related data, fast lookup by key, real-world data
Example	(1, 2, 3, "a")	{"name": "Ali", "age": 20}

Dictionaries

1. What is a Dictionary?

- A dictionary stores data in key : value pairs.
- It is mutable (can be changed).
- It is written inside curly braces { }.

Example:

```
d = {"name": "Ali", "age": 20, "course": "BSc CS"}
print(d)
```

Output: {'name': 'Ali', 'age': 20, 'course': 'BSc CS'}

2. Accessing & Modifying

- Use key to access values.
- Use key to add, modify or delete items.

```
d = {"name": "Ali", "age": 20}
print(d["name"]) # access value
d["city"] = "Akola" # add new key-value
d["age"] = 21 # modify value
print(d)
```

Output:

```
Ali
{'name': 'Ali', 'age': 21, 'city': 'Akola'}
```

3. Dictionary Operations

- keys(), values(), items()
- get(), update(), pop(), clear()
- len()

```
d = {"a": 1, "b": 2, "c": 3}
print(d.keys()) # dict_keys(['a', 'b', 'c'])
print(d.values()) # dict_values([1, 2, 3])
print(d.items()) # dict_items([('a', 1), ('b', 2), ('c', 3)])
print(len(d)) # 3
```

Output:

```
dict_keys(['a', 'b', 'c'])
dict_values([1, 2, 3])
dict_items([('a', 1), ('b', 2), ('c', 3)])
3
```



When to Use What?

Use a Tuple when:

- ✓ You need a fixed collection of items.
- ✓ You want to prevent data from being changed.
- ✓ You are returning multiple values from a function.

Use a Dictionary when:

- ✓ You need to store data with a meaningful key.
- ✓ You need to quickly look up values.
- ✓ You want to modify the data.

Tuples keep things fixed...
Dictionaries give you flexibility!

Tuple Methods and zip() Function

Tuples are ordered, immutable collections. They have very few built-in methods, but the most useful one is the `count()` and `index()` method. The `zip()` function is a handy built-in function used to combine multiple iterables.

Tuple Methods

Tuples have only two main built-in methods: `count()` and `index()`.

1. count() method

- Returns the number of times a specified value appears in the tuple.
- **Syntax:** `tuple.count(value)`

Example:

```
t = (10, 20, 30, 20, 40, 20)
print(t.count(20))    # Output: 3
print(t.count(50))    # Output: 0
```

Here, 20 appears 3 times in the tuple and 50 does not appear, so it returns 0.

2. index() method

- Returns the index of the first occurrence of a specified value.
- **Syntax:** `tuple.index(value)`

Example:

```
t = (10, 20, 30, 20, 40)
print(t.index(20))    # Output: 1
print(t.index(40))    # Output: 4
```

The first 20 is at index 1. The value 40 is at index 4. If the value is not found, it raises a `ValueError`.



Note: Tuples do not have methods like `append()`, `remove()`, etc. Because they are immutable (cannot be changed).

zip() Function

The `zip()` function is used to combine multiple iterables (like lists, tuples, or strings) element by element. It returns an iterator of tuples.

Syntax:

```
zip(iterable1, iterable2, ...)
```

Example 1: Using zip() with two lists

```
names = ['Aman', 'Riya', 'Sahil']
ages = [21, 19, 22]
result = zip(names, ages)
print(list(result))    # Output: [('Aman', 21), ('Riya', 19), ('Sahil', 22)]
```

Example 2: Using zip() with three lists

```
id = [101, 102, 103]
names = ['Aman', 'Riya', 'Sahil']
scores = [85, 90, 78]
result = zip(id, names, scores)
print(list(result))    # Output: [(101, 'Aman', 85), (102, 'Riya', 90), (103, 'Sahil', 78)]
```

Example 3: Unzipping a list of tuples

```
pairs = [(1, 'a'), (2, 'b'), (3, 'c')]
x, y = zip(*pairs)    # * unpacks the tuples
print(x)              # Output: (1, 2, 3)
print(y)              # Output: ('a', 'b', 'c')
```

★ Key Points:

- `zip()` stops when the shortest iterable is exhausted.
- It returns an iterator, so you can convert it to a list, tuple, etc.
- It is commonly used in loops, function arguments, and data processing.

Sets and Set Methods

A set is an **unordered** collection of **unique** elements.

What is a Set?

- A set is a collection of items, just like a list, but it is:
 - Unordered (no fixed position)
 - Does not allow duplicate elements
 - Written using curly braces { } or the set() function

Creating a Set

1. Using curly braces {}

```
fruits = {'apple', 'banana', 'cherry'} # set of strings
```

2. Using set() constructor

```
numbers = set([1, 2, 3, 4]) # set of integers
```

Example

```
s = {1, 2, 3, 4, 5}
print(type(s)) # <class 'set'>

# Duplicate elements are ignored
s = {1, 2, 2, 3, 4}
print(s) # {1, 2, 3, 4}
```

Accessing Elements

- Sets are unordered, so we cannot access elements by index.
- We can only use a loop to iterate through the set.

```
for item in s:
    print(item)
```

Set Operators

- Union (|) → all unique elements
- Intersection (&) → common elements
- Difference (-) → elements in first set
- Symmetric difference (^) → elements in either set, but not both

Set Methods

Sets have many built-in methods to perform operations on sets.

Method	Description	Example
add()	Adds an element to the set.	<pre>s = {1, 2, 3} s.add(4) # {1, 2, 3, 4}</pre>
update()	Adds multiple elements (from a list, tuple, set, etc.).	<pre>s = {1, 2, 3} s.update([4, 5]) # {1, 2, 3, 4, 5}</pre>
remove()	Removes an element. Raises KeyError if not found.	<pre>s = {1, 2, 3} s.remove(2) # {1, 3}</pre>
discard()	Removes an element. No error if not found.	<pre>s = {1, 2, 3} s.discard(5) # No error</pre>
pop()	Removes and returns an arbitrary element (since sets are unordered).	<pre>s = {1, 2, 3} item = s.pop() # item is 1 (or 2 or 3)</pre>
clear()	Removes all elements from the set.	<pre>s = {1, 2, 3} s.clear() # set()</pre>
copy()	Returns a shallow copy of the set.	<pre>s2 = s.copy()</pre>
len()	Returns the number of elements in the set.	<pre>s = {1, 2, 3, 4} len(s) # 4</pre>

★ Important Points

- Sets do not support indexing or slicing.
- Elements in a set must be hashable (immutable), e.g., numbers, strings, tuples.
- Lists, dictionaries and other mutable types cannot be added to a set.
- To use a list, convert it to a tuple or frozenset.

Quick Example

```
s = {1, 2, 3}
s.add(4)
s.update([5, 6])
print(s) # {1, 2, 3, 4, 5, 6}
s.discard(10) # No error
s.remove(3)
print(s) # {1, 2, 4, 5, 6}
```



Key Takeaway: Sets are useful when you need to work with **unique elements** and perform **mathematical set operations** easily.

Traversing Sets and Frozen Sets

Sets are **unordered** collections of **unique** elements. Let's see how to traverse them and what are frozen sets.

Traversing Sets

- Traversing means visiting each element in the set, one by one.
- Since sets are unordered, the elements can appear in any order.
- We use a loop (for loop) to traverse a set.

Example 1: Using for loop

```
fruits = {'apple', 'banana', 'cherry'}
for fruit in fruits:
    print(fruit)
```

Output (order may vary):

```
banana
apple
cherry
```

} (order is not fixed)

Example 2: Using enumerate()

```
colors = {'red', 'green', 'blue'}
for index, color in enumerate(colors):
    print(index, color)
```

Output (example):

```
0 green
1 red
2 blue
```

(order may vary)

Example 3: Using while loop

```
nums = {'10', '20', '30'}
i = 0
while i < len(nums):
    print(nums[i])
    i += 1
```

Output (example):

```
10
20
30
```

(order may vary)



Key Points:

- You can traverse a set using for loop, while loop, or enumerate().
- The order of elements is not guaranteed.
- Sets do not support indexing, so we cannot access elements by index.

Frozen Sets ❄️

- A frozen set is an immutable (unchangeable) version of a set.
- It is created using the frozenset() function.
- Since it is immutable, it cannot be modified (no add, remove, etc.).
- It can be used as a key in a dictionary and as an element in another set.

Creating a Frozen Set

```
fset = frozenset([1, 2, 3, 4])
print(fset)
```

Output:

```
frozenset({1, 2, 3, 4})
```

Trying to Modify (will give an error)

```
fset = frozenset([1, 2, 3])
fset.add(4) # Error
```

Output:

```
AttributeError:
'frozenset' object has no
attribute 'add'
```

Using Frozen Set as a Dictionary Key

```
my_dict = {frozenset({1, 2}): 'value'}
print(my_dict[frozenset({1, 2})])
```

Output:

```
value
```

Using Frozen Set in Another Set

```
s = {frozenset({1, 2}), frozenset({3, 4})}
print(s)
```

Output:

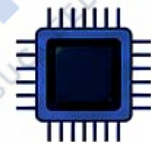
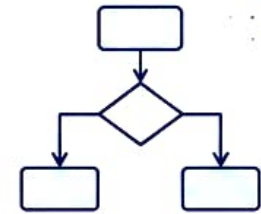
```
{frozenset({3, 4}),
frozenset({1, 2})}
```



Important Notes:

- frozenset() creates an immutable set.
- You cannot add, remove or modify elements.
- It can be used as a dictionary key or a set element.
- Useful when you need a set that should not change.

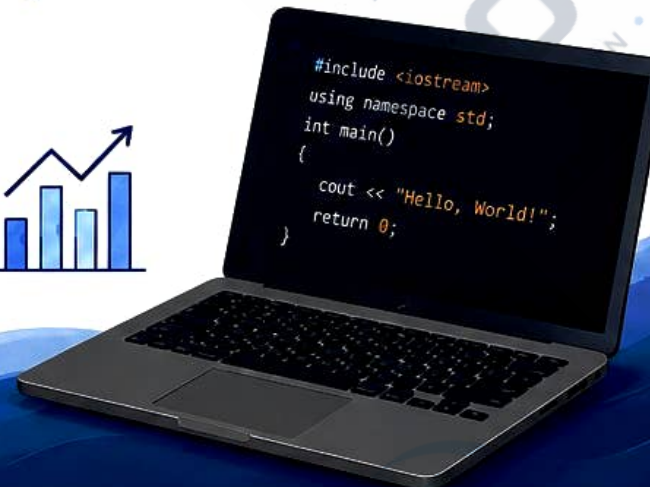
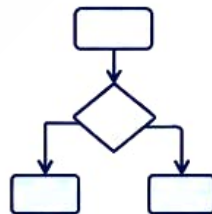
1010101010
0101010101
1010101010
0101010101



Unit - III



10110
01001
10101



Types of Files

Files are used to store data on a computer. Different types of files store different kinds of data and have different extensions.

1. Text Files



Extensions:

.txt, .log, .csv, .ini

- Store plain text (letters, numbers, symbols).
- Can be opened in any text editor.
- Example: notes.txt

2. Source Code Files



Extensions:

.c, .cpp, .java, .py, .html

- Contain program code written in a programming language.
- Used by developers to build software.
- Example: program.c

3. Image Files



Extensions:

.jpg, .jpeg, .png, .gif, .bmp, .tiff

- Store pictures and graphics.
- Can be viewed using image viewers.
- Example: photo.jpg

4. Audio Files



Extensions:

.mp3, .wav, .aac, .wma, .ogg

- Store sound and music.
- Played using media players.
- Example: song.mp3

5. Video Files



Extensions:

.mp4, .avi, .mkv, .mov, .wmv

- Store moving images and sound.
- Played using video players.
- Example: movie.mp4

6. Document Files



Extensions:

.doc, .docx, .pdf, .odt

- Store formatted text, tables, etc.
- Used for reports, letters, etc.
- Example: report.docx

7. Compressed Files



Extensions:

.zip, .rar, .7z, .tar

- Store multiple files in a smaller size.
- Used for easy sharing and storage.
- Example: files.zip

8. Executable Files



Extensions:

.exe, .msi, .app

- Run or install a program.
- Contains machine code.
- Example: setup.exe



Quick Summary

Type	Common Extensions	What it Stores	Example
Text	.txt, .log, .csv	Plain text	notes.txt
Source Code	.c, .py, .java	Program code	main.py
Image	.jpg, .png, .gif	Pictures/graphics	photo.jpg
Audio	.mp3, .wav, .aac	Sound	song.mp3
Video	.mp4, .avi, .mkv	Moving images + sound	movie.mp4
Document	.doc, .pdf, .odt	Formatted text	report.docx
Compressed	.zip, .rar, .7z	Multiple files (compressed)	files.zip
Executable	.exe, .msi, .app	Run/install a program	setup.exe



Key Points

- The extension (like .txt, .jpg, etc.) tells us what type of file it is.
- Different file types are used for different purposes (text, images, audio, video, etc.).
- Some files can be opened with common apps, while others need specific software.
- Compressed files help in saving space and faster transfer.
- Executable files can be run to install or use a program.

Right file type = Right use = Better work!

Creating & Reading Text Data

Text data is plain text (like a .txt file) and can be created, written to, read, and processed in Python using built-in file handling functions.

Creating & Writing Text Data

To create a text file and write data into it, we use the `open()` function with mode 'w' (write) or 'a' (append).

1. Create a new text file and write data

```
# Create a new file and write text
with open("notes.txt", "w") as file:
    file.write("Hello, Python!\n")
    file.write("This is a text file.")
```

- 'w' mode creates a new file or overwrites existing content.
- `write()` writes text to the file.

2. Append data to an existing file

```
with open("notes.txt", "a") as file:
    file.write("\nAppending a new line.")
```

- 'a' mode appends data to the end of the file.
- If the file doesn't exist, it will be created.

3. Using `write()` and `writelines()`

```
# Using write()
with open("data.txt", "w") as file:
    file.write("Line 1\nLine 2\n")
```

```
# Using writelines()
lines = ["Apple\n", "Banana\n", "Cherry\n"]
with open("fruits.txt", "w") as file:
    file.writelines(lines)
```

- `write()` writes a single string.
- `writelines()` writes a list of strings.
- Each string should end with '\n' (new line).

Reading Text Data

To read data from a text file, we use the `open()` function with mode 'r' (read).

1. Read the whole file

```
with open("notes.txt", 'r') as file:
    content = file.read()
    print(content)
```

- `read()` reads the entire file as one string.

2. Read line by line

```
with open("notes.txt", 'r') as file:
    for line in file:
        print(line, end="") # end="" avoids extra new line
```

- The file object can be looped to read line by line.
- `readline()` reads one line at a time.

3. Read lines into a list

```
with open("notes.txt", "r") as file:
    lines = file.readlines()
    print(lines)
```

- `readlines()` returns a list of all lines in the file.

Example:



notes.txt

File content:

Python is easy.
File handling is useful.
Keep learning!



Output (using `readlines()`):

```
['Python is easy.\n',
'File handling is useful.\n',
'Keep learning!']
```



Key Points

- Use `open()` to open a file. Always close the file or use a `with` statement.
- Modes: 'r' (read), 'w' (write), 'a' (append).
- Text files store data as plain text.
- `read()`, `readline()`, `readlines()`, `write()`, `writelines()` are useful methods.



Common File Modes

Mode	Use
'r'	Read (default mode)
'w'	Write (creates new file or overwrites)
'a'	Append (adds to existing file)



Use `with` statement for automatic closing of the file.

```
with open("file.txt", 'r') as f:
    # work with the file
```

File Methods for Reading / Writing

Files are used to store data permanently. Python provides different methods to read from and write to files easily.



File Methods for Reading

These methods are used to read data from a file.

Method	Description	Example
read()	• Reads the entire content of the file as a string.	<pre>f = open('notes.txt', 'r') content = f.read() print(content)</pre>
readline()	• Reads one line from the file. • Includes the newline character (\n).	<pre>f = open('notes.txt', 'r') line = f.readline() print(line)</pre>
readlines()	• Reads all lines and returns a list of strings.	<pre>f = open('notes.txt', 'r') lines = f.readlines() print(lines)</pre>
__iter__()	• Allows you to loop through the lines of a file.	<pre>f = open('notes.txt', 'r') for line in f: print(line)</pre>
tell()	• Returns the current position of the file pointer.	<pre>f = open('notes.txt', 'r') print(f.tell())</pre>
seek()	• Changes the current position of the file pointer.	<pre>f = open('notes.txt', 'r') f.seek(0) # go to start print(f.read())</pre>



Important Points (Reading)

- Files must be opened in read mode ("r") to read data.
- The read() method returns the whole content as a string.
- readline() returns one line, readlines() returns a list of lines.
- Use a loop (for line in f) to read line by line easily.



File Methods for Writing

These methods are used to write data to a file.

Method	Description	Example
write()	• Writes a string to the file. • Does not add a newline character automatically.	<pre>f = open('notes.txt', 'w') f.write("Hello Python!")</pre>
writelines()	• Writes a list of strings to the file. • Does not add newlines, so you must include them.	<pre>lines = ["Line 1\n", "Line 2\n"] f = open('notes.txt', 'w') f.writelines(lines)</pre>
flush()	• Forces the buffered data to be written to the file.	<pre>f.write("Data") f.flush()</pre>
close()	• Closes the file. • Frees up system resources.	<pre>f.close()</pre>



Example: Writing to a File

```
f = open('data.txt', 'w') # open file in write mode
a = ["Python", "is", "fun"]
f.writelines([word + "\n" for word in a])
f.close()
```

Content of data.txt:
Python
is
fun



Key Takeaways

- read() – reads entire file
- readline() – reads one line
- readlines() – reads all lines (list)
- write() – writes a string
- writelines() – writes list of strings
- flush() – forces data to be written
- close() – closes the file
- Use "r", "w", "a", "r+" etc. modes when opening a file.

Reading & Writing Binary Files

Binary files store data in binary format (0s and 1s) instead of plain text.
In Python, we use the **'b'** mode when opening a file to read or write binary data.

Key Points

- Binary files can store any type of data (e.g., images, audio, video, serialized objects).
- Use **'rb'** mode to read a binary file.
- Use **'wb'** mode to write a binary file.
- Use **'ab'** mode to append to a binary file.
- Data is read/written as bytes (not strings).
- Common method: read(), write(), and close().

1. Writing a Binary File

To write binary data, open the file in **'wb'** mode and use the write() method.

```
# Writing a binary file
data = b"Hello, Binary!" # bytes data
with open("example.bin", "wb") as f:
    f.write(data)
print("Binary file written successfully!")
```



Binary Data



example.bin

(written in binary mode)

2. Reading a Binary File

To read binary data, open the file in **'rb'** mode and use the read() method. It returns bytes.

```
# Reading a binary file
with open("example.bin", "rb") as f:
    data = f.read() # data is in bytes
print(data)
print(type(data))
```

Output (example):

```
b'Hello, Binary!'
<class 'bytes'>
```

Reading and Writing a Binary File (Example: Image File)

You can also copy a binary file (like an image) using Python.

```
# Copy an image (binary file)
with open("source.jpg", "rb") as f1:
    data = f1.read()
with open("copy.jpg", "wb") as f2:
    f2.write(data)
print("Image copied successfully!")
```



source.jpg
(binary file)



copy.jpg
(binary file)

Important Methods

Method	Use
open(filename, 'rb')	Open file for reading (binary)
open(filename, 'wb')	Open file for writing (binary)
open(filename, 'ab')	Open file for appending (binary)
read()	Read bytes from file
write(data)	Write bytes to file
close()	Close the file

Summary



- Binary files store data in 0s and 1s.
- Use **'rb'** and **'wb'** modes for reading and writing binary files.
- Always handle binary data as bytes, not strings.
- Useful for working with images, audio, video, and serialized objects (like pickle).

Binary files
= Real-world data
in Python!

Pickle Module

The `Pickle` module in Python is used to **serialize** and **deserialize** Python objects. It converts a Python object into a **byte stream** (so it can be stored in a file or sent over a network) and reconstructs the object back from the byte stream.

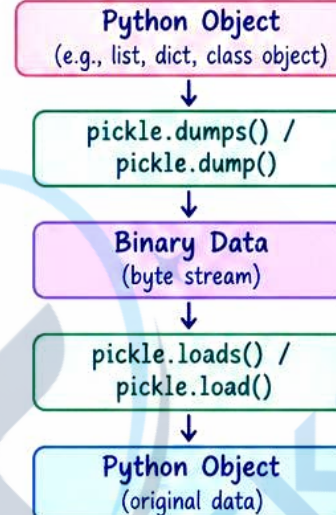
Key Points

- Pickle is a built-in module in Python.
- It is used to **serialize** (convert to bytes) and **deserialize** (convert back to object).
- It can handle most Python objects like numbers, strings, lists, dictionaries, tuples, sets, and user-defined classes.
- The serialized data is stored in a binary format (not human-readable).
- It is mainly used for storing data, caching, and inter-process communication.

Important Functions

Function	Description
<code>pickle.dump(obj, file)</code>	Serializes obj and writes it to a file.
<code>pickle.load(file)</code>	Reads the binary data from a file and reconstructs the object.
<code>pickle.dumps(obj)</code>	Serializes obj and returns a bytes object.
<code>pickle.loads(bytes_obj)</code>	Deserializes a bytes object back to a Python object.
<code>pickle.HIGHEST_PROTOCOL</code>	Uses the latest protocol version for serialization.

How it Works?



Example 1: Writing (Serializing) to a File

```
import pickle
# Data to store
data = {'name': 'Alice', 'age': 22, 'skills': ['Python', 'SQL']}

# Write to a binary file
with open('data.pkl', 'wb') as f:
    pickle.dump(data, f)
print("Data successfully written to file!")
```

✓ wb = write binary mode

Example 2: Reading (Deserializing) from a File

```
import pickle

# Read data from binary file
with open('data.pkl', 'rb') as f:
    data = pickle.load(f)

print("Data read from file:", data)
```

✓ rb = read binary mode

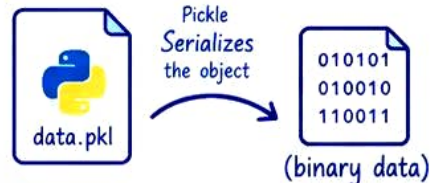
Example with a Class

```
import pickle
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

# Create object
s = Student("Shivam", 20)
# Serialize
with open('student.pkl', 'wb') as f:
    pickle.dump(s, f)
# Deserialize
with open('student.pkl', 'rb') as f:
    s1 = pickle.load(f)
print(s1.name, s1.age) # Output: Shivam 20
```

Applications

- Storing complex data structures.
- Caching data for faster access.
- Inter-process communication.
- Saving machine learning models (e.g., scikit-learn).
- Used in web frameworks and game development.



Summary

- Pickle converts Python objects to a binary format.
- Use `pickle.dump()` and `pickle.load()` for files.
- Use `pickle.dumps()` and `pickle.loads()` for bytes.
- Works with most Python objects (except (e.g., open files, sockets, etc.).
- Useful for data storage, caching and communication.



Reading & Writing CSV Files



A CSV (Comma Separated Values) file stores tabular data in plain text format, where each line represents a record and values are separated by commas.

Key Points

- CSV files are text files with values separated by commas (,).
- Python provides the built-in **csv module** to read and write CSV files.
- Use **open()** to open the file.
- Common functions: **reader()**, **writer()**, **DictReader()**, **DictWriter()**.
- Data is read/written as strings (convert if needed).
- Use **newline=''** in **writer()** to avoid extra blank lines on some systems.

1. Reading a CSV File

Use **csv.reader()** to read a CSV file line by line.

```
import csv

with open('students.csv', 'r', newline='') as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

Example CSV file (students.csv)

Name	Age	Course
Alice	20	BCA
Bob	21	B.Sc
Charlie	22	B.Com

Output

```
['Name', 'Age', 'Course']
['Alice', '20', 'BCA']
['Bob', '21', 'B.Sc']
['Charlie', '22', 'B.Com']
```

CSV File Format

- Values are separated by commas.
- Each line represents a record.
- The first line is usually the header (column names).
- Special characters (like commas in text) can be handled using quoting (quotechar='"').

Name	Age	Course
Alice	20	BCA
Bob	21	B.Sc
Charlie	22	B.Com

This is how a CSV file looks in a text editor.

2. Writing a CSV File

Use **csv.writer()** to write data to a CSV file.

```
import csv

data = [
    ['Name', 'Age', 'Course'],
    ['Alice', 20, 'BCA'],
    ['Bob', 21, 'B.Sc'],
    ['Charlie', 22, 'B.Com']
]

with open('output.csv', 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerows(data)
```

- ✓ Data is written in CSV format with commas and a new line after each row.

Using DictReader() and DictWriter()

These are useful when working with dictionaries (instead of lists).

Reading with DictReader

```
import csv
with open('students.csv', 'r', newline='') as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row['Name'], row['Age'], row['Course'])
```

Writing with DictWriter

```
import csv
data = [ {'Name': 'Alice', 'Age': 20, 'Course': 'BCA'},
         {'Name': 'Bob', 'Age': 21, 'Course': 'B.Sc'} ]

with open('output.csv', 'w', newline='') as f:
    fieldnames = ['Name', 'Age', 'Course']
    writer = csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader() # writes column names
    writer.writerows(data)
```

Common CSV Module Functions

Function	Description
csv.reader()	Reads data from a CSV file.
csv.writer()	Writes data to a CSV file.
csv.DictReader()	Reads CSV as dictionaries (uses header row).
csv.DictWriter()	Writes CSV using dictionaries (need fieldnames).
open()	Opens the file in read or write mode.

Summary

- Use **csv.reader()** to read CSV files.
- Use **csv.writer()** to write CSV files.
- For dictionary-based access, use **DictReader()** and **DictWriter()**.
- Always open files with **newline=''** when writing.
- CSV files are simple, lightweight, and widely used for data exchange.





os and os.path Modules



The **os** module provides a way of using operating system dependent functionality.
The **os.path** module provides common **pathname** manipulations.

1. os Module



Key Points

- The **os** module provides functions to interact with the operating system (files, directories, environment, etc.).
- It is platform dependent (works on Windows, Linux, macOS).
- Commonly used for file/directory operations, environment variables, and process management.
- Imported using: `import os`

Important Functions

Function	Description
<code>os.getcwd()</code>	Returns the current working directory.
<code>os.chdir(path)</code>	Changes the current working directory.
<code>os.listdir(path)</code>	Lists files and directories in a folder.
<code>os.mkdir(path)</code>	Creates a new directory.
<code>os.remove(path)</code>	Deletes a file.
<code>os.rmdir(path)</code>	Removes an empty directory.
<code>os.environ</code>	Accesses environment variables.



Example: Using os Module

```
import os
print(os.getcwd())      # current working directory
os.chdir('D:/Python')   # change directory
print(os.listdir())     # list files and folders
```



Remember

- Use **os** for general OS-related operations.
- Use **os.path** for path and file name manipulations.

2. os.path Module



Key Points

- The **os.path** module provides functions for manipulating file paths.
- It helps in splitting, joining, checking and working with paths in a platform-independent way.
- It is always used with the **os** module (i.e., `import os.path`).

Important Functions

Function	Description
<code>os.path.join(a, b)</code>	Joins path components.
<code>os.path.split(path)</code>	Splits the path into directory and file name.
<code>os.path.basename(path)</code>	Returns the file name from a path.
<code>os.path.dirname(path)</code>	Returns the directory name from a path.
<code>os.path.exists(path)</code>	Checks if a path exists.
<code>os.path.isfile(path)</code>	Checks if a path is a file.
<code>os.path.isdir(path)</code>	Checks if a path is a directory.
<code>os.path.abspath(path)</code>	Returns the absolute path.
<code>os.path.splitext(path)</code>	Splits the extension from a file name.



Example: Using os.path Module

```
import os.path
path = 'D:/Python/example.txt'
print(os.path.basename(path))  # example.txt
print(os.path.dirname(path))    # D:/Python
print(os.path.exists(path))    # True/False
```

Quick Difference

os Module	os.path Module
Provides OS-level functions.	Provides path manipulation functions.
Used for file/directory operations, environment variables, etc.	Used for splitting, joining and checking paths.
Examples: <code>os.getcwd()</code> , <code>os.listdir()</code>	Examples: <code>os.path.join()</code> , <code>os.path.exists()</code>



In Short

- **os** → works with the operating system.
- **os.path** → works with file paths.
- Both are important for file handling and directory management in Python.





Regular Expressions in Python

Regular expressions (regex) are patterns used to search, match and manipulate text.



Remember:

- Special characters help in pattern matching.
- Regex methods are used to apply these patterns in Python.

1. Special Characters in Regular Expressions

Special characters are symbols that have a special meaning in regex patterns.

Character	Meaning	Example	Matches
.	Matches any single character (except newline).	<code>r'a.b'</code>	<code>acb, alb, a@b ...</code>
^	Matches the start of a string (or line with MULTILINE).	<code>r'^Hello'</code>	<code>Hello, Hello123 ...</code>
\$	Matches the end of a string (or line with MULTILINE).	<code>r'world\$'</code>	<code>world, myworld ...</code>
*	Matches 0 or more repetitions (of the previous character).	<code>r'a*'</code>	<code>'', a, aa, aaa ...</code>
+	Matches 1 or more repetitions (of the previous character).	<code>r'a+'</code>	<code>a, aa, aaa ...</code> (not empty string)
?	Matches 0 or 1 occurrence (of the previous character).	<code>r'colou?r'</code>	<code>color, colour ...</code>
{n}	Matches exactly n occurrences.	<code>r'a{3}'</code>	<code>aaa</code>
{n,}	Matches n or more occurrences.	<code>r'a{2,}'</code>	<code>aa, aaa, aaaa ...</code>
{n,m}	Matches between n and m times (inclusive).	<code>r'a{2,4}'</code>	<code>aa, aaa, aaaa</code>
[]	Matches any one character from the set.	<code>r'[aeiou]'</code>	<code>a, e, i, o, u</code>
[^]	Matches any one character not in the set.	<code>r'^[0-9]'</code>	<code>A, b, #, @ ...</code>
()	Groups pattern (for reuse or precedence).	<code>r'(ab)+'</code>	<code>ab, abab, ababab ...</code>
	OR operator (matches left or right).	<code>r'cat dog'</code>	<code>cat, dog</code>
\	Escapes a special character (to treat it as a normal character).	<code>r'\.'</code>	<code>.</code> (dot character)

Some Useful Examples

- | | |
|--|--|
| • <code>r'\d'</code> → matches any digit (0-9) | • <code>r'\s'</code> → matches whitespace (space, tab, etc.) |
| • <code>r'\D'</code> → matches any non-digit | • <code>r'\S'</code> → matches non-whitespace |
| • <code>r'\w'</code> → matches any word character (a-z, A-Z, 0-9, _) | • <code>r'^\w+\$'</code> → whole string is word characters |
| • <code>r'\W'</code> → matches any non-word character | • <code>r'[a-zA-Z]'</code> → matches any letter |

2. Regular Expression Methods

Python's `re` module provides several methods to work with regular expressions.

```
import re
```

1. re.match()

Checks if the pattern matches at the beginning of the string.

```
text = "Hello123 World"
re.match(r'Hello', text)
# Match object (not None)
```

2. re.search()

Searches for the pattern anywhere in the string.

```
text = "Hello123 World"
re.search(r'\d+', text)
# Match object (finds 123)
```

3. re.findall()

Returns all non-overlapping matches as a list.

```
text = "abc123xyz456"
re.findall(r'\d+', text)
# ['123', '456']
```

4. re.finditer()

Returns an iterator of match objects (with position).

```
text = "abc123xyz"
for match in re.finditer(r'\d+', text):
    print(match.group(), match.start())
# Output: 123 3
```

5. re.sub()

Replaces matches with a new string.

```
text = "apple banana apple"
re.sub(r'apple', 'orange', text)
# 'orange banana orange'
```

6. re.split()

Splits the string by the pattern.

```
text = "one,two;three"
re.split(r'[;,]', text)
# ['one', 'two', 'three']
```

7. re.compile()

Compiles a regex pattern for reuse (better performance).

```
pattern = re.compile(r'\d+')
pattern.findall("a1b2c3")
# ['1', '2', '3']
```

8. re.fullmatch()

Checks if the whole string matches the pattern.

```
text = "12345"
re.fullmatch(r'\d+', text)
# Match object (not None)
```

Flags (Optional)

Used to modify the behavior of the regex.

- `re.IGNORECASE` → case-insensitive matching
- `re.MULTILINE` → `^` and `$` match start/end of each line
- `re.DOTALL` → `.` matches newline too
- `re.VERBOSE` → allows whitespace and comments in pattern

Example with Flags

```
text = "Hello\nWorld"
re.findall(r'hello', text, re.IGNORECASE | re.MULTILINE)
# ['Hello']
```



Practice with different patterns and methods to get better at regex!



Python Regular Expressions - Named Groups

Named groups help you give meaningful names to parts of your pattern, so you can easily access them in your code.

Why use Named Groups?

- Makes your regex more readable.
- Access groups by name (not just number).
- Useful when your pattern has many groups.
- Reduces confusion in large patterns.

1. What are Named Groups?

Named groups are created using `(?P<name>...)`. You can access the matched value using the group name (not the index).

`(?P<name>pattern)`

Example:

```
r'(?P<first>\w+)\s+(?P<last>\w+)'
```

Here, `first` and `last` are the names of the groups.

2. Example - Extracting Name and Email

Let's extract a person's name and email from a string using named groups.

```
import re
text = "Name: Saquib Khan, Email: saquib34@gmail.com"
pattern = r"Name: (?P<name>\w+\s*\w+),\s+Email: (?P<email>\S+)"
match = re.search(pattern, text)
if match:
    print("Name:", match.group('name'))
    print("Email:", match.group('email'))
```

Output:

```
Name: Saquib Khan
Email: saquib34@gmail.com
```

3. Another Example - Date Parsing

Extract day, month and year from a date using named groups.

```
import re
date = "Date: 25-08-2026"
pattern = r"Date: (?P<day>\d{2})-(?P<month>\d{2})-(?P<year>\d{4})"
match = re.search(pattern, date)
if match:
    print("Day:", match.group('day'))
    print("Month:", match.group('month'))
    print("Year:", match.group('year'))
```

Output:

```
Day: 25
Month: 08
Year: 2026
```

4. Common Named Group Syntax

Pattern	Meaning	Example
<code>(?P<name>...)</code>	Creates a named group	<code>(?P<name>\d+)</code>
<code>(?P=name)</code>	Backreference to a named group	<code>(?P=name)</code>
<code>(?P<name>...)</code>	Access by name (in code)	<code>match.group('name')</code>
<code>(?P<name>...)</code>	Use in <code>findall()</code> (returns dicts)	<code>re.findall(pattern, text)</code>
<code>(?P<name>...)</code>	Use in <code>sub()</code> (replacement)	<code>re.sub(pattern, repl, text)</code>

5. Using Named Groups with `findall()`

`findall()` returns a list of dictionaries when using named groups.

```
import re
text = "John 25, Mary 30, Alex 28"
pattern = r"(?P<name>\w+) (?P<age>\d+)"
matches = re.findall(pattern, text)
print(matches)
```

Output:

```
[{'name': 'John', 'age': '25'},
 {'name': 'Mary', 'age': '30'},
 {'name': 'Alex', 'age': '28'}]
```

6. Real-Life Example - URL Parsing

Extract protocol, domain and path from a URL.

```
import re
url = "https://www.example.com/path/page"
pattern = r"(?P<protocol>\w+)://(?P<domain>[^\s/]+)(?P<path>/?.*)"
match = re.search(pattern, url)
if match:
    print("Protocol:", match.group('protocol'))
    print("Domain:", match.group('domain'))
    print("Path:", match.group('path'))
```

Output:

```
Protocol: https
Domain: www.example.com
Path: /path/page
```

7. Quick Recap

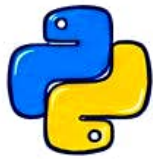
- ✓ Named groups use `(?P<name>...)`.
- ✓ Access values with `group('name')`.
- ✓ Can be used with `search()`, `findall()`, `sub()`, etc.
- ✓ Makes regex patterns cleaner and easier to maintain.

8. Quick Reference

Task	Syntax / Example
Create named group	<code>(?P<name>pattern)</code>
Access group by name	<code>match.group('name')</code>
Find all (dicts)	<code>re.findall(pattern, text)</code>
Use in <code>sub()</code>	<code>re.sub(pattern, repl, text)</code>
Backreference (named)	<code>(?P=name)</code>

Named Groups
= Clean Code + Easy Access
+ Less Confusion





Python

Regular Expressions with glob Module

Regular expressions (re) are used for pattern matching in strings.

The glob module is used for file name pattern matching (like wildcards).

Regex = search in text
glob = search in files

1. What is Regular Expression?

- A regular expression (regex) is a sequence of characters that forms a search pattern.
- Used to find, match, replace, or split text.
- Works with the re module in Python.

```
import re
# import the regular expression module
```

2. Common Regex Patterns

Pattern	Meaning	Example (matches)
.	Any single character	a.c → abc, a1c, a-c
^	Start of string	^Hello → Hello world
\$	End of string	world\$ → hello world
*	0 or more of previous	ab* → a, ab, abb, abbb
+	1 or more of previous	ab+ → ab, abb, abbb
?	0 or 1 of previous	colou?r → color, colour
{n}	Exactly n times	a{3} → aaa
{n,}	At least n times	a{2,} → aa, aaa, aaaa
{n,m}	Between n and m times	a{1,3} → a, aa, aaa
[abc]	Any one of the characters	[ae] → a, e
[a-z]	Range of characters	[a-z] → a, b, c, ..., z
[^abc]	Not any of the characters	[^0-9] → non-digit
	OR operator	cat dog → cat, dog
()	Group (for precedence)	(ab)+ → ab, abab, ...
\	Escape special character	\. → matches a literal dot

Important Tips

- Use raw strings (r'...') to avoid escape issues.
- Use re.IGNORECASE for case-insensitive matching.
- Use re.MULTILINE (^) and \$ match at start/end of each line.
- Use re.DOTALL (.) matches newlines too.
- Escape special characters (e.g. \, *, \?).

3. Useful re Functions

re.search()
Searches for the first match (anywhere).

```
re.search(r'\d+', 'abc123')
# Match: 123
```

re.match()
Checks if the pattern matches at the beginning.

```
re.match(r'hello', 'hello world')
# Match: hello
```

re.findall()
Returns all matches as a list.

```
re.findall(r'\d+', 'a1b22c3')
# ['1', '22', '3']
```

re.finditer()
Returns an iterator of match objects.

```
for m in re.finditer(r'\d+', 'a1b22c3'):
    print(m.group())
# 1 22 3
```

re.sub()
Replaces matches with a new string.

```
re.sub(r'\d+', '#', 'a1b22c3')
# 'a#b##c#'
```

re.split()
Splits the string by the pattern.

```
re.split(r'\s+', 'apple, banana, mango')
# ['apple', 'banana', 'mango']
```

re.compile()
Compiles a regex pattern (better performance).

```
pattern = re.compile(r'\d+')
pattern.findall('a1b2c3')
# ['1', '2', '3']
```

Example: Finding Emails

```
import re
text = "Contact us at info@abc.com or support@test.org"
emails = re.findall(r'[\w.-]+@[\w.-]+\.\w+', text)
print(emails)
# ['info@abc.com', 'support@test.org']
```

4. What is glob Module?

- The glob module is used to find files and directories using wildcard patterns.
- It matches the file names (not the content).
- It works like a simple pattern matching tool (similar to * and ?).

```
import glob
# import the glob module
```

Key Points

- Uses wildcards (*, ?, []).
- Returns a list of matching file/directory names.
- Works with paths.
- Useful for batch file operations.

5. Wildcards in glob

Wildcard	Meaning	Example
*	Matches 0 or more characters	*.txt → all .txt files
?	Matches exactly one character	file?.txt → file1.txt, file2.txt
[abc]	Matches any one of the characters	file[12].txt → file1.txt, file2.txt
[a-z]	Matches any character in range	file[a-c].txt → filea.txt, fileb.txt
[!abc]	Matches any character not in set	file[!0-9].txt → filea.txt, fileb.txt

6. Examples of glob

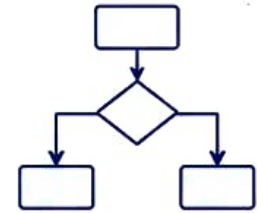
glob.glob('*.txt')	# All .txt files in current directory
glob.glob('*.py')	# All .py files in current directory
glob.glob('data/*')	# All files and folders inside data/
glob.glob('data/*.csv')	# All .csv files inside data/
glob.glob('data/?readme.txt')	# Matches data/areadme.txt, data/breadme.txt, etc.
glob.glob('**/*.py', recursive=True)	# All .py files in subdirectories

7. Compare: re vs glob

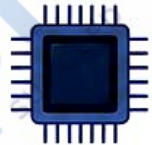
Feature	re (Regular Expressions)	glob (File Pattern Matching)
Purpose	Text pattern matching	File name pattern matching
Pattern syntax	More powerful (regex syntax)	Simple wildcards (*, ?, [], etc.)
Used for	Strings, text, data	Files and directories
Module	re	glob
Example	re.findall(r'\d+', 'abc123')	glob.glob('*.txt')

Different tools,
Same goal...
Find what you need!

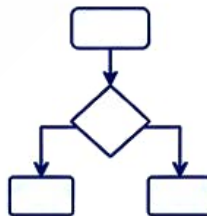
1010101010
0101010101
1010101010
0101010101



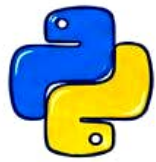
Unit - IV



10110
01001
10101



```
#include <iostream>
using namespace std;
int main()
{
    cout << "Hello, World!";
    return 0;
}
```



Python Object-Oriented Programming (OOP)

Classes & Objects

A class is a blueprint. An object is an instance of that class.
Classes help us organize code and create reusable structure.



Remember:

- Class = blueprint / template
- Object = instance of class
- Class variables = shared by all objects
- Instance variables = unique to each object
- Methods = functions inside a class

1. What is a Class?

- A class is a user-defined blueprint for creating objects.
- It defines attributes (data) and methods (functions).
- It helps in organizing and reusing code.

Example:

```
class Car:
    brand = "Toyota" # class variable (shared)

    def __init__(self, model, color):
        self.model = model # instance variable
        self.color = color

    def show(self):
        print(f"Car: {self.brand} {self.model}, Color: {self.color}")
```

2. Creating Objects

- An object is created from a class using the class name followed by parentheses (.).
- The `__init__()` method is a constructor, which initializes the object.

Example:

```
# Create objects
car1 = Car("Corolla", "White")
car2 = Car("Honda", "Black")

# Use the method
car1.show()
car2.show()
```

Output:

```
Car: Toyota Corolla, Color: White
Car: Toyota Honda, Color: Black
```

3. Instance Variables vs Class Variables

- Class variables are shared by all objects.
- Instance variables are unique to each object.

Example:

```
class Student:
    school = "SGBAU" # class variable

    def __init__(self, name, marks):
        self.name = name # instance variable
        self.marks = marks

s1 = Student("Saqib", 85)
s2 = Student("Ali", 90)
print(s1.school) # SGBAU
print(s2.school) # SGBAU
print(s1.name) # Saqib
print(s2.name) # Ali
```

Output:

```
SGBAU
SGBAU
Saqib
Ali
```

Key Point:

- school → class variable
- name, marks → instance variables

4. Defining Methods

- Methods are functions defined inside a class.
- They help the object perform actions.

Example:

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width

r = Rectangle(5, 3)
print("Area:", r.area())
```

Output:

```
Area: 15
```



self
self refers to the current object.

5. Types of Methods

Type	Description	Example
Instance Method	Works with object's instance data.	<code>def area(self): ...</code>
Class Method	Works with class data (using <code>@classmethod</code>).	<code>@classmethod</code> <code>def from_string(cls, s): ...</code>
Static Method	Doesn't use class or instance data.	<code>@staticmethod</code> <code>def add(a, b): ...</code>

Example (Class Method & Static Method):

```
class Math:
    @classmethod
    def add(cls, a, b):
        return a + b

    @staticmethod
    def square(n):
        return n * n

print(Math.add(5, 3)) # 8
print(Math.square(4)) # 16
```

6. Accessing Object Attributes

- You can access or modify attributes using dot (.) operator.
- You can also delete an attribute using `del`.

Example:

```
car = Car("BMW", "Red")
print(car.brand) # Toyota (class variable)
print(car.model) # BMW (instance variable)
car.color = "Blue" # modify attribute
print(car.color) # Blue
del car.model # delete attribute
print(car.__dict__) # show remaining attributes
```

Output:

```
Toyota
BMW
Blue

{'color': 'Blue'}
```

7. Constructors & Destructors

- `__init__()` → called when object is created.
- `__del__()` → called when object is about to be destroyed (not commonly used).

Example:

```
class Demo:
    def __init__(self):
        print("Object created")

    def __del__(self):
        print("Object destroyed")
```

Output:

```
Object created
Object destroyed

(when object is deleted)
```

8. Real-Life Example

Think of a class as a form (template) and objects as filled forms.



Class
(Blueprint)



Object 1



Object 2



Object 3

Build your own classes,
create your own world
in Python!





Creating Classes in Python | Creating Objects

A **class** is a blueprint. An **object** is an instance of that class.

1. What is a Class?

- A class is a user-defined blueprint for creating objects.
- It defines attributes (data) and methods (functions).
- It helps in organizing and reusing code.

2. Syntax of a Class

```
class ClassName:  
    # class attributes (optional)  
    attribute = value  
  
    def method_name(self, parameters):  
        # method body  
        return result
```

Key Points:

- class keyword is used to create a class.
- self refers to the current object.
- Methods are functions inside a class.
- Attributes can be class or instance level.

3. Example - Creating a Class

```
class Student:  
    # class attribute  
    school = "SGBAU"  
  
    def __init__(self, name, marks):  
        self.name = name # instance attribute  
        self.marks = marks  
  
    def display(self):  
        print(f"Name: {self.name}, Marks: {self.marks}, School: {Student.school}")
```

Explanation:

- __init__() is a constructor (called when object is created).
- self.name and self.marks are instance attributes.
- display() is a method.

4. More Examples

Class with Multiple Methods

```
class Car:  
    def __init__(self, brand, model):  
        self.brand = brand  
        self.model = model  
  
    def show(self):  
        print(f"Car: {self.brand} {self.model}")
```

Class with Class Method

```
class Employee:  
    company = "SGBAU"  
  
    def __init__(self, name, id):  
        self.name = name  
        self.id = id  
  
    @classmethod  
    def get_company(cls):  
        return cls.company
```

5. Creating Objects

- An object is an instance of a class.
- We create an object by calling the class name with (.).
- The object can access attributes and methods of the class.

Syntax

```
object_name = ClassName(parameters)
```

Key Points:

- Each object has its own copy of instance attributes.
- Objects can access methods using dot (.) operator.
- Multiple objects can be created from the same class.

6. Example - Creating Objects

```
class Student:  
    school = "SGBAU"  
    def __init__(self, name, marks):  
        self.name = name  
        self.marks = marks  
    def display(self):  
        print(f"Name: {self.name}, Marks: {self.marks}, School: {Student.school}")  
  
# Creating objects  
s1 = Student("Saqib", 85)  
s2 = Student("Ali", 90)  
  
# Accessing methods  
s1.display()  
s2.display()
```

Output:

```
Name: Saqib, Marks: 85,  
School: SGBAU  
Name: Ali, Marks: 90,  
School: SGBAU
```

7. Accessing Attributes

```
print(s1.name) # Saqib  
print(s1.marks) # 85  
print(s1.school) # SGBAU (class attribute)
```

Output:

```
Saqib  
85  
SGBAU
```

8. Summary

- ✓ Class = blueprint (defines structure).
- ✓ Object = instance (real-world representation).
- ✓ Use __init__() for initialization.
- ✓ Use self to access attributes and methods.
- ✓ You can create multiple objects from one class.

Class (Blueprint)

```
Student  
• name  
• marks  
• display()
```

Objects (Instances)

```
s1  
name: Saqib  
marks: 85  
  
s2  
name: Ali  
marks: 90
```

Same class → Multiple objects

Build
Your Own
World in
Python!





Python

Constructor Method & Classes with Multiple Objects

The constructor method (`__init__`) is used to initialize the object.
A class can have multiple objects, and each object has its own copy of attributes.



Remember:

- `__init__()` is the constructor.
- It runs automatically when an object is created.
- Each object has its own data.

1. Constructor Method (`__init__`)

What is a Constructor?

- A constructor is a special method used to initialize the attributes of a class.
- In Python, the constructor method is `__init__()`.
- It is called automatically when an object is created.



Key Points:

- Name: `__init__()`
- Used to initialize attributes.
- Runs automatically.
- No return value (usually).

Example: Using Constructor Method

```
class Student:
    def __init__(self, name, marks):
        self.name = name # instance attribute
        self.marks = marks

    def display(self):
        print(f"Name: {self.name}, Marks: {self.marks}")

# Creating an object (constructor is called automatically)
s1 = Student("Saqib", 85)
s1.display()
```

Output:

Name: Saqib,
Marks: 85

How `__init__` Works?

- 1 `s1 = Student("Saqib", 85)` → creates an object.
- 2 `__init__(self, "Saqib", 85)` → initializes attributes.
- 3 `self.name` and `self.marks` are set for `s1`.
- 4 `display()` shows the values.

Without Constructor (for comparison)

```
class Student:
    def set_data(self, name, marks):
        self.name = name
        self.marks = marks

s1 = Student()
s1.set_data("Saqib", 85)
```

Here, we have to call a separate method to set the values.



Remember:

Constructor makes object creation easy and ensures that the object is always in a valid state.

2. Classes with Multiple Objects

What does it mean?

- You can create multiple objects from the same class.
- Each object has its own copy of instance variables.
- Changes in one object do not affect the other object.



Key Points:

- Same class → multiple objects.
- Each object has its own memory.
- Attributes are independent.
- Methods are shared (same code).

Example: Multiple Objects

```
class Car:
    def __init__(self, brand, model, color):
        self.brand = brand
        self.model = model
        self.color = color

    def display(self):
        print(f"Brand: {self.brand}, Model: {self.model}, Color: {self.color}")

# Creating multiple objects
car1 = Car("Toyota", "Corolla", "White")
car2 = Car("Honda", "Civic", "Black")
car3 = Car("BMW", "X5", "Blue")

# Accessing methods for each object
car1.display()
car2.display()
car3.display()
```

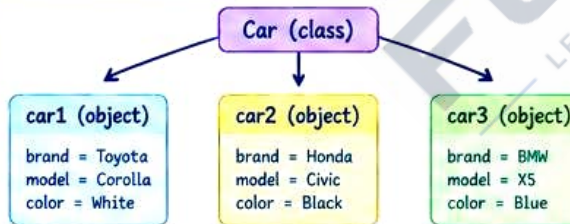
Output:

Brand: Toyota,
Model: Corolla,
Color: White

Brand: Honda,
Model: Civic,
Color: Black

Brand: BMW,
Model: X5,
Color: Blue

Visual Representation (Memory View)



Each object has its own values, but uses the same class (same methods).

Example with User Input

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print(f"Name: {self.name}, Age: {self.age}")

# Multiple objects
p1 = Person("Ali", 20)
p2 = Person("Zara", 22)
p1.display()
p2.display()
```



Key Takeaway: The constructor (`__init__`) helps in initializing objects, and multiple objects allow you to create independent instances of the same class.



Class Attributes vs Data Attributes



In Python, attributes are variables that hold values. They can be defined at the class level or at the instance level. Let's see the difference!

Class Attributes

Class attributes are variables that are defined inside the class but outside any method. They are **shared by all instances** of the class.

Key Points:

- Defined directly in the class (outside methods).
- Shared by all objects (instances) of the class.
- Used for values that are common to all objects (e.g., count, version).
- Accessed using the class name or the instance.

Data Attributes

Data attributes are variables that are defined inside the constructor (`__init__`) or any instance method. They are **specific to each object** and can have different values for different instances.

Key Points:

- Defined inside `__init__()` or instance methods.
- Unique for each object (instance).
- Used to store data specific to that object (e.g., name, age).
- Accessed using the instance (`self`).

Feature	Class Attributes	Data Attributes
Where defined?	Inside the class, outside methods	Inside <code>__init__()</code> or instance methods
Shared or individual?	Shared by all instances	Individual for each instance
Accessed using	Class name or instance	Instance (<code>self</code>)
Common use	Common values (e.g., count, version)	Object-specific data (e.g., name, age)

Example: Class Attributes

```
class Employee:
    company = "ABC Corp" # class attribute

    def __init__(self, name, age):
        self.name = name # data attribute
        self.age = age # data attribute

emp1 = Employee("Alice", 25)
emp2 = Employee("Bob", 30)

print(Employee.company) # Output: ABC Corp (shared)
print(emp1.company) # Output: ABC Corp
print(emp2.company) # Output: ABC Corp
```

Example: Data Attributes

```
class Student:
    school = "Green Valley" # class attribute (common)

    def __init__(self, name, age):
        self.name = name # data attribute
        self.age = age # data attribute

s1 = Student("Riya", 20)
s2 = Student("Aman", 22)

print(s1.name) # Output: Riya
print(s2.name) # Output: Aman
print(s1.school) # Output: Green Valley (same for all)
```



In Short:

Class attributes are **shared**, while data attributes are **unique** to each object.

Encapsulation and Inheritance

Both are important OOP concepts in Python that help in building secure, reusable and maintainable code.

Encapsulation



What is Encapsulation?

Encapsulation is the process of wrapping data (attributes) and the methods (functions) that operate on that data into a single unit called a class. It also restricts direct access to some of the class's components using access modifiers (like **private**).

Key Points

- Bundles data and methods into a single unit (**class**).
- Hides internal details using access modifiers (e.g., **__**).
- Protects data from unauthorized access.
- Provides controlled access through getter and setter methods.
- Achieved using private members (e.g., **__var**) and methods.

Example: Encapsulation in Python

```
class Student:
    def __init__(self, name, age):
        self.__name = name # private attribute
        self.__age = age

    def get_name(self):
        return self.__name
    def get_age(self):
        return self.__age
    def set_age(self, age):
        if age > 0:
            self.__age = age
        else:
            print("Age must be positive")

# Creating an object
s1 = Student("Riya", 20)
print(s1.get_name()) # Access using getter
print(s1.get_age()) # Access using getter
# print(s1.__name) # Error! (private attribute)
```

Class

Private Data
(**__name**, **__age**)

Methods
(**get_name()**,
get_age(), **set_age()**)

Data and methods
are bundled together
and hidden from
direct access.

Inheritance



What is Inheritance?

Inheritance is a mechanism in which a new class (**derived class**) acquires the properties and behaviors of an existing class (**base class**). It promotes code reusability and reduces redundancy.

Key Points

- Reuses code from an existing class.
- Helps in creating a hierarchical relationship between classes.
- Supports the concept of **method overriding**.
- Improves maintainability and reduces duplication.
- In Python, inheritance is done using the **class** keyword and parentheses (**()**).

Example: Inheritance in Python

```
# Base class
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return "Animal sound"

# Derived class
class Dog(Animal):
    def speak(self):
        return "Bark!"

# Creating an object
d = Dog("Buddy")
print(d.name) # Inherited from Animal
print(d.speak()) # Overridden method
```

Animal
(base class)

__init__()
speak()

Dog
(derived class)

__init__()
speak() ← overridden

Dog inherits the properties
and methods of **Animal**.



Quick Comparison

Focus

Main Goal

Achieved Using

Benefit

Encapsulation

Hiding data and bundling with methods

Security and controlled access

Access modifiers (private), getters/setters

Data protection, better control

Inheritance

Reusing code from an existing class

Code reusability and hierarchical relationship

class keyword and inheritance syntax

Less code, easier maintenance

Encapsulation
protects data,
Inheritance
builds relationships!



Polymorphism

Same method
Different behaviour
!

Polymorphism means “many forms”. It allows the same method (or operator) to behave differently depending on the object.



Key Points

- Polymorphism allows the same method name to be used for different types of objects.
- It is achieved through **method overriding** (in inheritance) and **operator overloading** (in Python, using special methods like `__add__`, `__sub__`, etc.).
- It makes the code more **flexible**, reusable and easy to maintain.
- In Python, polymorphism is achieved mainly by method overriding and operator overloading.

Types of Polymorphism

1. Method Overriding



- A **subclass** provides its own version of a method that is already defined in the parent class.
- It works at runtime.
- Commonly used in inheritance.

2. Operator Overloading



- The same operator (e.g., `+`) behaves differently for different data types.
- In Python, it is achieved using special methods (e.g., `__add__`, `__sub__`, etc.).

Example 1: Method Overriding

```
class Animal:
    def sound(self):
        return "Animal sound"

class Dog(Animal):
    def sound(self):
        return "Bark"

class Cat(Animal):
    def sound(self):
        return "Meow"
```

```
a = Animal()
d = Dog()
c = Cat()
print(a.sound()) # Animal sound
print(d.sound()) # Bark
print(c.sound()) # Meow
```

Same method name (`sound()`) behaves differently for different objects.

Example 2: Operator Overloading

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Vector(self.x + other.x,
                       self.y + other.y)

v1 = Vector(2, 3)
v2 = Vector(4, 5)
v3 = v1 + v2 # __add__ is called
print(v3.x, v3.y) # 6 8
```

The `+` operator works differently for Vector objects (adds the **coordinates**).

Real-Life Analogy

Same method name: `make_sound()`



Same method, different behaviour based on the object.

Benefits

- ✓ Makes code more flexible and reusable.
- ✓ Improves maintainability and readability.
- ✓ Supports dynamic method binding (runtime behavior).
- ✓ Helps in building scalable and modular programs.



Summary

Polymorphism in Python is achieved through method overriding and operator overloading. It lets the same method or operator behave differently for different objects, making the code **more flexible**, reusable and easier to maintain.

One interface
Many forms
!